

Deep Learning Formula Cheat Sheet

deep-learning-formula-cheatsheet

Contents

1	Notation and Tensor Shapes	1
2	Logistic Regression and Binary Classification	3
3	Forward Propagation	5
4	Activation Functions	6
5	Loss and Cost Functions	7
6	Backpropagation	9
7	Gradient Descent and Optimization	11
8	Initialization	15
9	Regularization	17
10	Batch Normalization	19
11	Convolutional Neural Networks	21
12	Classic CNN Architectures	24
13	Object Detection and YOLO	25
14	Face Recognition and Neural Style Transfer	26
15	RNN / GRU / LSTM	28
16	Word Embeddings and Language Models	33
17	Seq2Seq, Beam Search, and Attention	37
18	Transformer and Self-Attention	41
19	ML Strategy Formula Appendix	46
20	Shape Reference Tables	49

1 Notation and Tensor Shapes

Global convention

m : number of examples in a batch, L : number of layers,
 n_x : input dimension, n_y : output dimension,
 $n^{[l]}$: number of units in layer l , $n^{[0]} = n_x$.

Use: Column-wise batch notation.

Notes: Example index: $\langle i \rangle$. Layer index: $[l]$. Sequence index: $\langle t \rangle$. Mini-batch index: $\{k\}$.

Batch matrices

$$X = A^{[0]} = \begin{bmatrix} | & | & \dots & | \\ x^{(1)} & x^{(2)} & \dots & x^{(m)} \\ | & | & & | \end{bmatrix} \in \mathbb{R}^{n_x \times m}, \quad Y \in \mathbb{R}^{n_y \times m}.$$

Shapes: $x^{(i)} \in \mathbb{R}^{n_x \times 1}$, $y^{(i)} \in \mathbb{R}^{n_y \times 1}$.

Use: Vectorized training examples.

Dense layer parameters

$$\begin{aligned} W^{[l]} &\in \mathbb{R}^{n^{[l]} \times n^{[l-1]}}, \\ b^{[l]} &\in \mathbb{R}^{n^{[l]} \times 1}, \\ Z^{[l]}, A^{[l]} &\in \mathbb{R}^{n^{[l]} \times m}. \end{aligned}$$

Use: Layer-wise shape contract.

Notes: $b^{[l]}$ is broadcast across the m batch columns.

Dense layer forward pass

$$\begin{aligned} Z^{[l]} &= W^{[l]} A^{[l-1]} + b^{[l]}, \\ A^{[l]} &= g^{[l]}(Z^{[l]}). \end{aligned}$$

Shapes: $W^{[l]} A^{[l-1]} : (n^{[l]} \times n^{[l-1]})(n^{[l-1]} \times m) = n^{[l]} \times m$.

Use: Default affine-activation block.

Gradient shapes

$$\begin{aligned} dZ^{[l]}, dA^{[l]} &\in \mathbb{R}^{n^{[l]} \times m}, \\ dW^{[l]} &\in \mathbb{R}^{n^{[l]} \times n^{[l-1]}}, \\ db^{[l]} &\in \mathbb{R}^{n^{[l]} \times 1}. \end{aligned}$$

Use: Backpropagation dimension checks.

Elementwise operations

$$U \odot V, \quad g(U), \quad \log U, \quad U^2$$

Shapes: All elementwise operands must be equal shape or broadcastable.

Use: Activation, loss, dropout, and gates.

Common vectors

$$\mathbf{1}_m \in \mathbb{R}^{1 \times m}, \quad \sum_{i=1}^m U_{:,i} = U \mathbf{1}_m^T.$$

Use: Batch sums and bias gradients.

Mini-batch notation

$$X^{\{k\}} \in \mathbb{R}^{n_x \times m_k}, \quad Y^{\{k\}} \in \mathbb{R}^{n_y \times m_k}.$$

Use: Mini-batch k with size m_k .

Notes: Do not use t for mini-batches; t is reserved for sequence time steps and optimizer steps.

CNN notation

H, W : spatial height and width,
 C : channels,
 f : filter size,
 p : padding,
 s : stride.

Shapes: Main convention: NHWC. Single image: $H \times W \times C$. Batch: $m \times H \times W \times C$.

Use: CNN shape formulas.

Notes: PyTorch commonly uses NCHW: $m \times C \times H \times W$. Keep mapping in the shape reference appendix.

Sequence notation

$$x^{(t)} \in \mathbb{R}^{n_x \times m}, \quad a^{(t)} \in \mathbb{R}^{n_a \times m}, \quad \hat{y}^{(t)} \in \mathbb{R}^{n_y \times m}.$$

Use: RNN, GRU, LSTM, and sequence models.

Notes: T_x is input length. T_y is output length.

Transformer notation

$$X \in \mathbb{R}^{T \times d_{\text{model}}}, \quad Q \in \mathbb{R}^{T_q \times d_k}, \quad K \in \mathbb{R}^{T_k \times d_k}, \quad V \in \mathbb{R}^{T_v \times d_v}.$$

Use: Self-attention and cross-attention.

Notes: Section 18 starts with single-head notation, then adds batch and multi-head shape references.

Transformer batch and head references

$$Q \in \mathbb{R}^{m \times T_q \times d_k}, \quad Q \in \mathbb{R}^{m \times h \times T_q \times d_k}.$$

Use: Batch and multi-head shape checks.

Notes: The same batch/head convention applies to K and V with their sequence and value dimensions.

Probability notation

$$\hat{y}^{(i)} = P(y = 1 | x^{(i)}), \quad \hat{y}_k^{(i)} = P(y = k | x^{(i)}).$$

Use: Binary and multiclass outputs.

Parameter update notation

$$\theta \leftarrow \theta - \alpha d\theta.$$

Shapes: θ and $d\theta$ have identical shape.

Use: Generic first-order update rule.

2 Logistic Regression and Binary Classification

2.1 Assumptions and Shapes

Binary logistic setup

$$X \in \mathbb{R}^{n_x \times m}, \quad Y \in \mathbb{R}^{1 \times m}, \quad w \in \mathbb{R}^{n_x \times 1}, \quad b \in \mathbb{R}.$$
$$x^{(i)} \in \mathbb{R}^{n_x \times 1}, \quad y^{(i)} \in \{0, 1\}.$$

Use: Binary classification forward/backward pipeline.

Notes: b is broadcast across all m examples.

2.2 Forward

Single-example forward

$$z^{(i)} = w^\top x^{(i)} + b,$$
$$a^{(i)} = \hat{y}^{(i)} = \sigma(z^{(i)}).$$

Shapes: $z^{(i)}, a^{(i)}, \hat{y}^{(i)} \in \mathbb{R}$.

Use: Single-example probability output.

Vectorized forward

$$\begin{aligned} Z &= w^\top X + b, \\ A &= \sigma(Z). \end{aligned}$$

Shapes: $Z, A, Y \in \mathbb{R}^{1 \times m}$.

Use: Batch probability output.

2.3 Loss and Cost

Binary cross-entropy loss

$$\mathcal{L}(a^{(i)}, y^{(i)}) = - \left[y^{(i)} \log a^{(i)} + (1 - y^{(i)}) \log(1 - a^{(i)}) \right].$$

Shapes: $a^{(i)}, y^{(i)} \in \mathbb{R}$.

Use: Single-example binary loss.

Average cost

$$\begin{aligned} \mathcal{J}(w, b) &= \frac{1}{m} \sum_{i=1}^m \mathcal{L}(a^{(i)}, y^{(i)}). \\ \mathcal{J}(w, b) &= -\frac{1}{m} \sum_{i=1}^m \left[y^{(i)} \log a^{(i)} + (1 - y^{(i)}) \log(1 - a^{(i)}) \right]. \end{aligned}$$

Use: Average binary negative log-likelihood.

2.4 Backward

Vectorized gradients

$$\begin{aligned} dZ &= A - Y, \\ dw &= \frac{1}{m} X dZ^\top, \\ db &= \frac{1}{m} \sum_{i=1}^m dZ^{(i)}. \end{aligned}$$

Shapes: $dZ \in \mathbb{R}^{1 \times m}$, $dw \in \mathbb{R}^{n_x \times 1}$, $db \in \mathbb{R}$.

Use: Sigmoid plus binary cross-entropy gradients.

2.5 Parameter Update

Gradient descent update

$$\begin{aligned} w &\leftarrow w - \alpha dw, \\ b &\leftarrow b - \alpha db. \end{aligned}$$

Shapes: $w, dw \in \mathbb{R}^{n_x \times 1}$; $b, db, \alpha \in \mathbb{R}$.

Use: One optimization step.

2.6 Symbols and Shapes

Symbol	Shape / domain	Role
$x^{(i)}$	$\mathbb{R}^{n_x \times 1}$	single input column
$y^{(i)}$	$\{0, 1\}$	single binary label
X	$\mathbb{R}^{n_x \times m}$	input batch
Y	$\mathbb{R}^{1 \times m}$	label batch
w	$\mathbb{R}^{n_x \times 1}$	weight vector
b	$\mathbb{R}$	scalar bias
Z	$\mathbb{R}^{1 \times m}$	logits
A	$\mathbb{R}^{1 \times m}$	predictions
dZ	$\mathbb{R}^{1 \times m}$	logit gradient
dw	$\mathbb{R}^{n_x \times 1}$	weight gradient
db	$\mathbb{R}$	bias gradient
α	$\mathbb{R}_{>0}$	learning rate
m	$\mathbb{N}$	batch size
n_x	$\mathbb{N}$	input dimension

3 Forward Propagation

3.1 Assumptions and Dense-Layer Shapes

Layer input and parameters

$$A^{[0]} = X, \quad A^{[l-1]} \in \mathbb{R}^{n^{[l-1]} \times m}.$$

$$W^{[l]} \in \mathbb{R}^{n^{[l]} \times n^{[l-1]}}, \quad b^{[l]} \in \mathbb{R}^{n^{[l]} \times 1}.$$

Use: Dense layer shape contract.

Notes: $b^{[l]}$ is broadcast across the batch axis.

3.2 Layer-wise Forward Propagation

Affine-activation block

$$Z^{[l]} = W^{[l]} A^{[l-1]} + b^{[l]},$$

$$A^{[l]} = g^{[l]}(Z^{[l]}).$$

Shapes: $Z^{[l]}, A^{[l]} \in \mathbb{R}^{n^{[l]} \times m}$.

Use: Dense layer forward pass.

Shape check

$$(n^{[l]} \times n^{[l-1]}) \cdot (n^{[l-1]} \times m) \longrightarrow n^{[l]} \times m.$$

Use: Matrix multiplication compatibility.

3.3 Deep Network Composition

Recursive composition

$$A^{[0]} = X,$$

$$Z^{[l]} = W^{[l]} A^{[l-1]} + b^{[l]},$$

$$A^{[l]} = g^{[l]}(Z^{[l]}), \quad l = 1, \dots, L.$$

$$A^{[L]} = g^{[L]} \left(W^{[L]} A^{[L-1]} + b^{[L]} \right).$$

Use: Deep network forward pipeline.

3.4 Output Layer Choices

Task	Output activation	Output width	Output shape
Binary classification	$A^{[L]} = \sigma(Z^{[L]})$	$n^{[L]} = 1$	$\mathbb{R}^{1 \times m}$
Multi-class classification	$A^{[L]} = \text{softmax}(Z^{[L]})$	$n^{[L]} = K$	$\mathbb{R}^{K \times m}$
Regression	$A^{[L]} = Z^{[L]}$	$n^{[L]} = n_y$	$\mathbb{R}^{n_y \times m}$

3.5 Cache Notation

Layer cache

$$\text{cache}^{[l]} = (A^{[l-1]}, W^{[l]}, b^{[l]}, Z^{[l]}).$$

Use: Stored quantities for backpropagation.

3.6 Symbols and Shapes

Symbol	Shape / domain	Role
$A^{[0]}$	$\mathbb{R}^{n_x \times m}$	input batch
$A^{[l-1]}$	$\mathbb{R}^{n^{[l-1]} \times m}$	previous activation
$W^{[l]}$	$\mathbb{R}^{n^{[l]} \times n^{[l-1]}}$	layer weights
$b^{[l]}$	$\mathbb{R}^{n^{[l]} \times 1}$	layer bias
$Z^{[l]}$	$\mathbb{R}^{n^{[l]} \times m}$	pre-activation
$A^{[l]}$	$\mathbb{R}^{n^{[l]} \times m}$	activation
$g^{[l]}$	elementwise or output map	activation function
L	$\mathbb{N}$	number of layers
$n^{[l]}$	$\mathbb{N}$	units in layer l
m	$\mathbb{N}$	batch size

4 Activation Functions

4.1 Elementwise Hidden Activations

Activation	Formula	Derivative	Range	Common use
Sigmoid	$\sigma(z) = \frac{1}{1 + e^{-z}}$	$\sigma'(z) = \sigma(z)(1 - \sigma(z))$	$(0, 1)$	Gate / binary output
Tanh	$\tanh(z) = \frac{e^z - e^{-z}}{e^z + e^{-z}}$	$\frac{d}{dz} \tanh(z) = 1 - \tanh^2(z)$	$(-1, 1)$	Hidden state
ReLU	$\text{ReLU}(z) = \max(0, z)$	$\text{ReLU}'(z) = \begin{cases} 1, & z > 0 \\ 0, & z < 0 \end{cases}$	$[0, \infty)$	Hidden layer
Leaky ReLU	$\text{LeakyReLU}(z) = \max(\lambda_{\text{leak}} z, z)$	$\text{LeakyReLU}'(z) = \begin{cases} 1, & z > 0 \\ \lambda_{\text{leak}}, & z < 0 \end{cases}$	$\mathbb{R}$	Hidden layer

Use: Elementwise hidden-layer nonlinearities.

Notes: Derivatives at nondifferentiable points are implementation conventions.

4.2 Output Activations

Activation	Formula	Constraint	Common use
Sigmoid	$a = \sigma(z)$	$a = P(y = 1 x)$	Binary
Softmax	$\text{softmax}(z)_k = \frac{e^{z_k}}{\sum_{j=1}^K e^{z_j}}$	$a_k = P(y = k x), \sum_{k=1}^K a_k = 1$	Multi-class
Linear	$a = z$	$a \in \mathbb{R}^{n_y}$	Regression

4.3 Derivative Table

Softmax Jacobian

$$\frac{\partial a_i}{\partial z_j} = a_i(\delta_{ij} - a_j).$$

Shapes: $z, a \in \mathbb{R}^{K \times 1}$ for one example.

Use: Softmax derivative reference.

Symbol	Meaning	Shape / domain
z	scalar pre-activation or logit vector	$\mathbb{R}$ or $\mathbb{R}^{K \times 1}$
a	activation output	same leading shape as z
K	number of classes	$\mathbb{N}$
δ_{ij}	Kronecker delta	$\{0, 1\}$
λ_{leak}	leaky ReLU slope	$0 < \lambda_{\text{leak}} < 1$

4.4 Common Output-Loss Simplifications

Output activation	Loss	Logit gradient
Sigmoid	Binary cross-entropy	$dZ = A - Y$
Softmax	Categorical cross-entropy	$dZ = A - Y$

Shapes: A, Y, dZ share the output-layer batch shape.

Use: Output-layer backpropagation shortcut.

5 Loss and Cost Functions

5.1 Binary Cross-Entropy [Core]

Single-example binary loss

$$\mathcal{L}(a, y) = -[y \log a + (1 - y) \log(1 - a)].$$

Shapes: $a \in (0, 1), y \in \{0, 1\}$.

Use: Binary classification.

Batch binary cost

$$\mathcal{J} = \frac{1}{m} \sum_{i=1}^m \mathcal{L}(a^{(i)}, y^{(i)}).$$

Shapes: $A, Y \in \mathbb{R}^{1 \times m}$.

Use: Average binary negative log-likelihood.

5.2 Multiclass Cross-Entropy [Core]

Single-example multiclass loss

$$\mathcal{L}(\hat{y}, y) = - \sum_{k=1}^K y_k \log \hat{y}_k.$$

Shapes: $\hat{y}, y \in \mathbb{R}^{K \times 1}$, $\sum_{k=1}^K \hat{y}_k = 1$.

Use: Multi-class classification.

One-hot true class form

$$\mathcal{L}(\hat{y}, y) = - \log \hat{y}_c.$$

Shapes: $c \in \{1, \dots, K\}$.

Use: One-hot categorical target.

5.3 Mean Squared Error [Derived]

Single-example squared error

$$\mathcal{L}(\hat{y}, y) = \frac{1}{2} \|\hat{y} - y\|_2^2.$$

Shapes: $\hat{y}, y \in \mathbb{R}^{n_y \times 1}$.

Use: Regression.

Batch mean squared error

$$\mathcal{J} = \frac{1}{m} \sum_{i=1}^m \frac{1}{2} \left\| \hat{y}^{(i)} - y^{(i)} \right\|_2^2.$$

Shapes: $\hat{Y}, Y \in \mathbb{R}^{n_y \times m}$.

Use: Average regression objective.

5.4 Sequence Negative Log-Likelihood [Core]

Conditional sequence loss

$$\mathcal{L} = - \sum_{t=1}^{T_y} \log P \left(y^{(t)} \mid y^{(1)}, \dots, y^{(t-1)}, x \right).$$

Shapes: t is a sequence time-step index; T_y is output length.

Use: Sequence modeling.

5.5 Regularized Cost [Derived]

L2-regularized objective

$$\mathcal{J}_{\text{reg}} = \mathcal{J} + \frac{\lambda}{2m} \sum_{l=1}^L \left\| W^{[l]} \right\|_F^2.$$

Shapes: $W^{[l]} \in \mathbb{R}^{n^{[l]} \times n^{[l-1]}}$.

Use: L2-regularized objective.

5.6 Label Smoothing [Optional]

Smoothed target

$$y_{\text{smooth}} = (1 - \varepsilon)y + \frac{\varepsilon}{K}.$$

$$\mathcal{L} = - \sum_{k=1}^K (y_{\text{smooth}})_k \log \hat{y}_k.$$

Shapes: $y, y_{\text{smooth}}, \hat{y} \in \mathbb{R}^{K \times 1}$.

Use: Regularizes over-confident predictions.

5.7 Focal Loss [Optional]

Focal loss

$$\text{FL}(p_t) = -\alpha(1 - p_t)^\gamma \log(p_t).$$

Shapes: $p_t \in (0, 1)$; α, γ are scalar hyperparameters.

Use: Handles class imbalance in object detection.

5.8 Symbols

Symbol	Shape / domain	Role
$\mathcal{L}$	scalar	single-example or sequence loss
$\mathcal{J}$	scalar	batch average cost
a	$(0, 1)$	binary prediction
$\hat{y}$	$\mathbb{R}^{K \times 1}$ or $\mathbb{R}^{n_y \times 1}$	prediction vector
y	label domain	target
K	$\mathbb{N}$	number of classes
c	$\{1, \dots, K\}$	true class index
T_y	$\mathbb{N}$	output sequence length
λ	$\mathbb{R}_{\geq 0}$	L2 regularization strength
$\ W^{[l]}\ _F^2$	scalar	squared Frobenius norm
y_{smooth}	$\mathbb{R}^{K \times 1}$	smoothed target distribution
ε	scalar	label-smoothing strength
p_t	$(0, 1)$	focal-loss target probability
α, γ	scalar	focal-loss weights

6 Backpropagation

6.1 Backward Pipeline [Derived]

Dense-layer reverse flow

$$dA^{[l]} \rightarrow dZ^{[l]} \rightarrow dW^{[l]}, db^{[l]} \rightarrow dA^{[l-1]}.$$

Use: Reverse-mode differentiation for dense layers.

6.2 Output Layer Simplifications [Core]

Sigmoid output with binary cross-entropy

$$dZ^{[L]} = A^{[L]} - Y.$$

Shapes: $A^{[L]}, Y, dZ^{[L]} \in \mathbb{R}^{1 \times m}$.

Use: Binary output-layer gradient.

Softmax output with categorical cross-entropy

$$dZ^{[L]} = A^{[L]} - Y.$$

Shapes: $A^{[L]}, Y, dZ^{[L]} \in \mathbb{R}^{K \times m}$.

Use: Multi-class output-layer gradient.

Notes: Same compact form, different output dimension.

6.3 General Hidden-Layer Backpropagation [Core]

Activation gradient

$$dZ^{[l]} = dA^{[l]} \odot g'^{[l]}(Z^{[l]}).$$

Shapes: $dA^{[l]}, dZ^{[l]}, Z^{[l]} \in \mathbb{R}^{n^{[l]} \times m}$.

Use: Hidden-layer pre-activation gradient.

Previous-activation gradient

$$dA^{[l-1]} = (W^{[l]})^\top dZ^{[l]}.$$

$$W^{[l]} \in \mathbb{R}^{n^{[l]} \times n^{[l-1]}}, \quad dA^{[l-1]} \in \mathbb{R}^{n^{[l-1]} \times m}.$$

Use: Gradient passed to previous layer.

Shape check for $dA^{[l-1]}$

$$(W^{[l]})^\top dZ^{[l]} : (n^{[l-1]} \times n^{[l]}) \cdot (n^{[l]} \times m) \rightarrow n^{[l-1]} \times m.$$

Use: Backpropagation dimension check.

6.4 Parameter Gradients [Derived]

Weight gradient

$$dW^{[l]} = \frac{1}{m} dZ^{[l]} (A^{[l-1]})^\top.$$

Shapes: $dW^{[l]} \in \mathbb{R}^{n^{[l]} \times n^{[l-1]}}$.

Use: Dense-layer weight gradient.

Bias gradient

$$db^{[l]} = \frac{1}{m} \sum_{i=1}^m dZ^{[l](i)} \in \mathbb{R}^{n^{[l]} \times 1}.$$

Use: Dense-layer bias gradient.

Notes: $dZ^{[l](i)}$ is the i -th column of $dZ^{[l]}$.

Shape check for $dW^{[l]}$

$$dZ^{[l]} (A^{[l-1]})^\top : (n^{[l]} \times m) \cdot (m \times n^{[l-1]}) \rightarrow n^{[l]} \times n^{[l-1]}.$$

Use: Parameter-gradient dimension check.

6.5 Shape Check and Parameter Update Link [Derived]

Basic gradient descent update

$$\begin{aligned} W^{[l]} &\leftarrow W^{[l]} - \alpha dW^{[l]}, \\ b^{[l]} &\leftarrow b^{[l]} - \alpha db^{[l]}. \end{aligned}$$

Shapes: $W^{[l]}, dW^{[l]} \in \mathbb{R}^{n^{[l]} \times n^{[l-1]}}$; $b^{[l]}, db^{[l]} \in \mathbb{R}^{n^{[l]} \times 1}$.

Use: Basic gradient descent update.

6.6 Symbols and Shapes

Symbol	Meaning	Shape
$A^{[l-1]}$	previous activation	$\mathbb{R}^{n^{[l-1]} \times m}$
$Z^{[l]}$	pre-activation	$\mathbb{R}^{n^{[l]} \times m}$
$A^{[l]}$	activation	$\mathbb{R}^{n^{[l]} \times m}$
$W^{[l]}$	layer weights	$\mathbb{R}^{n^{[l]} \times n^{[l-1]}}$
$b^{[l]}$	layer bias	$\mathbb{R}^{n^{[l]} \times 1}$
$dA^{[l]}$	activation gradient	$\mathbb{R}^{n^{[l]} \times m}$
$dZ^{[l]}$	pre-activation gradient	$\mathbb{R}^{n^{[l]} \times m}$
$dW^{[l]}$	weight gradient	$\mathbb{R}^{n^{[l]} \times n^{[l-1]}}$
$db^{[l]}$	bias gradient	$\mathbb{R}^{n^{[l]} \times 1}$
$g'^{[l]}$	activation derivative	elementwise map
m	batch size	$\mathbb{N}$
$n^{[l]}$	units in layer l	$\mathbb{N}$
θ_i	scalar parameter	scalar
e_i	basis vector for θ_i	same shape as θ
g	backprop gradient	parameter shape
g_{approx}	numerical gradient	same shape as g
diff	gradient-check discrepancy	scalar

6.7 Gradient Checking [Optional]

Numerical gradient approximation

$$\frac{\partial \mathcal{J}}{\partial \theta_i} \approx \frac{\mathcal{J}(\theta + \epsilon e_i) - \mathcal{J}(\theta - \epsilon e_i)}{2\epsilon}.$$

Shapes: θ_i is one scalar parameter; e_i is the matching basis vector.

Use: Sanity check for backprop implementation.

Vector-form approximation

$$g_{\text{approx}} \approx \frac{\mathcal{J}(\theta + \epsilon) - \mathcal{J}(\theta - \epsilon)}{2\epsilon}.$$

Shapes: g_{approx} has the same shape as g .

Use: Numerical gradient vector.

Relative difference

$$\text{diff} = \frac{\|g - g_{\text{approx}}\|_2}{\|g\|_2 + \|g_{\text{approx}}\|_2}.$$

Use: Gradient-check discrepancy.

Notes: ϵ small; disable dropout / randomness.

7 Gradient Descent and Optimization

7.1 Basic Gradient Descent [Core]

Generic update

$$\theta \leftarrow \theta - \alpha \nabla_{\theta} \mathcal{J}(\theta).$$

Shapes: θ and $\nabla_{\theta}\mathcal{J}(\theta)$ have identical shape.

Use: Basic full-batch or mini-batch parameter update.

Layer-wise dense update

$$\begin{aligned} W^{[l]} &\leftarrow W^{[l]} - \alpha dW^{[l]}, \\ b^{[l]} &\leftarrow b^{[l]} - \alpha db^{[l]}. \end{aligned}$$

$$W^{[l]}, dW^{[l]} \in \mathbb{R}^{n^{[l]} \times n^{[l-1]}}, \quad b^{[l]}, db^{[l]} \in \mathbb{R}^{n^{[l]} \times 1}.$$

Use: Dense-layer parameter update.

7.2 Mini-batch Gradient Descent [Core]

Mini-batch

$$X^{\{k\}} \in \mathbb{R}^{n_x \times m_k}, \quad Y^{\{k\}} \in \mathbb{R}^{n_y \times m_k}.$$

Use: Mini-batch k with m_k examples.

Mini-batch cost and update

$$\mathcal{J}^{\{k\}} = \frac{1}{m_k} \sum_{i=1}^{m_k} \mathcal{L}(\hat{y}^{(i)}, y^{(i)}).$$

$$\theta \leftarrow \theta - \alpha \nabla_{\theta} \mathcal{J}^{\{k\}}(\theta).$$

Use: Mini-batch parameter update.

7.3 Exponentially Weighted Average [Optional]

EWA

$$\begin{aligned} v_t &= \beta v_{t-1} + (1 - \beta) q_t. \\ \hat{v}_t &= \frac{v_t}{1 - \beta^t}. \end{aligned}$$

Use: Smooth a sequence of noisy values.

Notes: q_t is the observed quantity; v_t is its exponentially weighted average.

Notes: Here t denotes optimizer/update step, not sequence time step or mini-batch index.

7.4 Momentum [Derived]

Generic momentum update

$$\begin{aligned} g_t &= \nabla_{\theta} \mathcal{J}_t(\theta_{t-1}). \\ u_t &= \beta u_{t-1} + (1 - \beta) g_t, \\ \theta_t &= \theta_{t-1} - \alpha u_t. \end{aligned}$$

Shapes: u_t, g_t, θ_t have identical parameter shape.

Use: Momentum update rule.

Notes: Uses course-style EWA scaling with $(1 - \beta)$.

Layer-wise momentum

$$\begin{aligned} u_{dW^{[l]},t} &= \beta u_{dW^{[l]},t-1} + (1 - \beta) dW_t^{[l]}, \\ u_{db^{[l]},t} &= \beta u_{db^{[l]},t-1} + (1 - \beta) db_t^{[l]}. \\ W_t^{[l]} &= W_{t-1}^{[l]} - \alpha u_{dW^{[l]},t}, \\ b_t^{[l]} &= b_{t-1}^{[l]} - \alpha u_{db^{[l]},t}. \end{aligned}$$

Shapes: $u_{dW^{[l]},t} \in \mathbb{R}^{n^{[l]} \times n^{[l-1]}}$; $u_{db^{[l]},t} \in \mathbb{R}^{n^{[l]} \times 1}$.

Use: Layer-wise momentum accumulators.

7.5 RMSProp [Derived]

Generic RMSProp update

$$\begin{aligned} g_t &= \nabla_{\theta} \mathcal{J}_t(\theta_{t-1}). \\ s_t &= \beta s_{t-1} + (1 - \beta) g_t^{\odot 2}, \\ \theta_t &= \theta_{t-1} - \alpha \frac{g_t}{\sqrt{s_t} + \epsilon}. \end{aligned}$$

Shapes: s_t, g_t, θ_t have identical parameter shape.

Use: Elementwise squared-gradient scaling.

Notes: RMSProp is shown without bias correction.

Layer-wise RMSProp

$$\begin{aligned} s_{dW^{[l]},t} &= \beta s_{dW^{[l]},t-1} + (1 - \beta) (dW_t^{[l]})^{\odot 2}, \\ s_{db^{[l]},t} &= \beta s_{db^{[l]},t-1} + (1 - \beta) (db_t^{[l]})^{\odot 2}. \\ W_t^{[l]} &= W_{t-1}^{[l]} - \alpha \frac{dW_t^{[l]}}{\sqrt{s_{dW^{[l]},t}} + \epsilon}, \\ b_t^{[l]} &= b_{t-1}^{[l]} - \alpha \frac{db_t^{[l]}}{\sqrt{s_{db^{[l]},t}} + \epsilon}. \end{aligned}$$

Shapes: $s_{dW^{[l]},t} \in \mathbb{R}^{n^{[l]} \times n^{[l-1]}}$; $s_{db^{[l]},t} \in \mathbb{R}^{n^{[l]} \times 1}$.

Notes: Squares, square roots, and divisions are elementwise.

7.6 Adam [Core]

Generic Adam update

$$\begin{aligned} g_t &= \nabla_{\theta} \mathcal{J}_t(\theta_{t-1}). \\ m_t &= \beta_1 m_{t-1} + (1 - \beta_1) g_t, \\ v_t &= \beta_2 v_{t-1} + (1 - \beta_2) g_t^{\odot 2}. \\ \hat{m}_t &= \frac{m_t}{1 - \beta_1^t}, \quad \hat{v}_t = \frac{v_t}{1 - \beta_2^t}. \\ \theta_t &= \theta_{t-1} - \alpha \frac{\hat{m}_t}{\sqrt{\hat{v}_t} + \epsilon}. \end{aligned}$$

Shapes: m_t, v_t, g_t, θ_t have identical parameter shape.

Use: Momentum plus RMSProp update.

Notes: In Adam, m_t is the first-moment accumulator; batch size remains m .

Layer-wise Adam accumulator shapes

$$m_{dW^{[l]}}, v_{dW^{[l]}} \in \mathbb{R}^{n^{[l]} \times n^{[l-1]}}, \quad m_{db^{[l]}}, v_{db^{[l]}} \in \mathbb{R}^{n^{[l]} \times 1}.$$

Use: Dense-layer Adam state shapes.

7.7 Learning Rate Decay [Optional]

Decay type	Formula
Inverse-time	$\alpha_t = \frac{\alpha_0}{1 + \lambda t}$
Exponential	$\alpha_t = \alpha_0 \gamma^t$
Step	$\alpha_t = \alpha_0 \gamma^{\lfloor t/r \rfloor}$

Use: Learning-rate schedule.

7.8 Hyperparameter Sampling (Log Scale) [Optional]

Log-scale learning-rate sample

$$\alpha = 10^r, \quad r \sim \text{Uniform}(a, b).$$

Shapes: $a, b, r \in \mathbb{R}; \alpha > 0$.

Use: Learning-rate search over orders of magnitude.

7.9 Gradient Clipping [Optional]

Elementwise clipping

$$g \leftarrow \text{clip}(g, -c, c).$$

Shapes: g is a gradient tensor; $c > 0$ is scalar.

Use: Prevent exploding gradients in RNNs / Transformers.

Norm clipping

$$g \leftarrow g \frac{c}{\max(c, \|g\|_2)}.$$

Shapes: g and clipped g have identical shape.

Use: Clip gradient norm to threshold c .

7.10 Optimizer Symbol Table

Symbol	Meaning	Shape / Type
θ_t	parameter value at step t	parameter shape
$\mathcal{J}$	cost function	scalar
g_t	gradient at step t	same shape as θ_t
q_t	generic quantity for EWA	scalar or tensor
α	learning rate	scalar
α_t	learning rate at step t	scalar
α_0	initial learning rate	scalar
β	EWA / Momentum / RMSProp decay	scalar in $[0, 1)$
β_1	Adam first-moment decay	scalar in $[0, 1)$
β_2	Adam second-moment decay	scalar in $[0, 1)$
ϵ	denominator stabilizer	small scalar
v_t	EWA value or Adam second moment	same shape as tracked quantity
u_t	Momentum accumulator	same shape as θ_t
s_t	RMSProp squared-gradient accumulator	same shape as θ_t
m_t	Adam first-moment accumulator	same shape as θ_t
t	optimizer/update step	$\mathbb{N}$
k	mini-batch index	$\mathbb{N}$
$X^{\{k\}}$	input mini-batch	$\mathbb{R}^{n_x \times m_k}$
$Y^{\{k\}}$	label mini-batch	$\mathbb{R}^{n_y \times m_k}$
m_k	mini-batch size	$\mathbb{N}$
λ	inverse-time decay rate	scalar
γ	multiplicative decay factor	scalar
r	step-decay interval	$\mathbb{N}$
a, b	log-scale sampling bounds	scalars
c	clipping threshold	positive scalar
g	generic gradient	parameter shape

8 Initialization

8.1 Parameter Shapes

Layer parameter shapes

$$W^{[l]} \in \mathbb{R}^{n^{[l]} \times n^{[l-1]}}, \quad b^{[l]} \in \mathbb{R}^{n^{[l]} \times 1}.$$

$$\text{fan_in} = n^{[l-1]}, \quad \text{fan_out} = n^{[l]}.$$

Use: Layer-wise initialization shape reference.

8.2 Zero Initialization

Zero parameters

$$W^{[l]} = 0, \quad b^{[l]} = 0.$$

Shapes: $W^{[l]} \in \mathbb{R}^{n^{[l]} \times n^{[l-1]}}$; $b^{[l]} \in \mathbb{R}^{n^{[l]} \times 1}$.

Use: Zero initialization reference.

Notes: Bias can be initialized to zero. Weight matrices should not all be initialized to zero in hidden layers.

8.3 Small Random Initialization

Gaussian small random weights

$$W^{[l]} \sim \mathcal{N}(0, \sigma^2), \quad b^{[l]} = 0.$$

Shapes: $W^{[l]} \in \mathbb{R}^{n^{[l]} \times n^{[l-1]}}$; $b^{[l]} \in \mathbb{R}^{n^{[l]} \times 1}$.

Use: Simple symmetry-breaking initialization.

Notes: σ is chosen small in simple networks.

8.4 Xavier / Glorot Initialization

Glorot / Xavier variance

$$\text{Var}(W^{[l]}) = \frac{2}{\text{fan_in} + \text{fan_out}}.$$

Use: Common initialization for tanh / sigmoid-style networks.

Xavier normal

$$W^{[l]} \sim \mathcal{N}\left(0, \frac{2}{\text{fan_in} + \text{fan_out}}\right).$$

Shapes: $W^{[l]} \in \mathbb{R}^{n^{[l]} \times n^{[l-1]}}$.

Use: Normal Xavier / Glorot initialization.

Xavier uniform

$$W^{[l]} \sim \mathcal{U}\left(-\sqrt{\frac{6}{\text{fan_in} + \text{fan_out}}}, \sqrt{\frac{6}{\text{fan_in} + \text{fan_out}}}\right).$$

Shapes: $W^{[l]} \in \mathbb{R}^{n^{[l]} \times n^{[l-1]}}$.

Use: Uniform Xavier / Glorot initialization.

Course simplified Xavier-style variance

$$\text{Var}(W^{[l]}) = \frac{1}{\text{fan_in}}.$$

Use: Common simplified Xavier-style convention.

Notes: $1/\text{fan_in}$ is a common simplified convention in some course notes.

8.5 He Initialization

He variance

$$\text{Var}(W^{[l]}) = \frac{2}{\text{fan_in}}.$$

Use: Common initialization for ReLU-style hidden layers.

He normal

$$W^{[l]} \sim \mathcal{N}\left(0, \frac{2}{\text{fan_in}}\right).$$

Shapes: $W^{[l]} \in \mathbb{R}^{n^{[l]} \times n^{[l-1]}}$.

Use: Normal He initialization.

He scaled random matrix

$$W^{[l]} = \sqrt{\frac{2}{n^{[l-1]}}} R^{[l]}, \quad R^{[l]} \sim \mathcal{N}(0, 1).$$

Shapes: $R^{[l]}, W^{[l]} \in \mathbb{R}^{n^{[l]} \times n^{[l-1]}}$.

Use: Standard-deviation form.

8.6 Bias Initialization

Bias initialization

$$b^{[l]} = 0, \quad b^{[l]} \in \mathbb{R}^{n^{[l]} \times 1}.$$

Use: Bias initialization.

Notes: Bias zero initialization does not cause hidden-unit symmetry when weights are randomly initialized.

8.7 Initialization Symbol Table

Symbol	Meaning	Shape / Type
$W^{[l]}$	layer weights	$\mathbb{R}^{n^{[l]} \times n^{[l-1]}}$
$b^{[l]}$	layer bias	$\mathbb{R}^{n^{[l]} \times 1}$
$n^{[l]}$	units in layer l	$\mathbb{N}$
$n^{[l-1]}$	units in previous layer	$\mathbb{N}$
fan_in	input width	$n^{[l-1]}$
fan_out	output width	$n^{[l]}$
σ^2	initialization variance	scalar
$R^{[l]}$	standard normal random matrix	$\mathbb{R}^{n^{[l]} \times n^{[l-1]}}$
$\mathcal{N}(0, \sigma^2)$	normal distribution	distribution
$\mathcal{U}(a, b)$	uniform distribution	distribution

9 Regularization

9.1 L2 Regularized Cost

L2-regularized objective

$$\mathcal{J}_{\text{reg}} = \mathcal{J} + \frac{\lambda}{2m} \sum_{l=1}^L \|W^{[l]}\|_F^2.$$

$$\|W^{[l]}\|_F^2 = \sum_{i,j} \left(W_{ij}^{[l]}\right)^2.$$

Use: L2-regularized objective.

Notes: Bias terms are usually not included in this penalty.

9.2 L2 Gradient Term

Regularized weight gradient

$$dW_{\text{reg}}^{[l]} = dW^{[l]} + \frac{\lambda}{m} W^{[l]}.$$

$$dW_{\text{reg}}^{[l]}, dW^{[l]}, W^{[l]} \in \mathbb{R}^{n^{[l]} \times n^{[l-1]}}.$$

Use: Gradient update with L2 penalty.

9.3 Weight Decay Form

Weight update with L2 term

$$W^{[l]} \leftarrow W^{[l]} - \alpha \left(dW^{[l]} + \frac{\lambda}{m} W^{[l]} \right).$$

Use: L2 penalty in gradient step.

Equivalent shrinkage form

$$W^{[l]} \leftarrow \left(1 - \alpha \frac{\lambda}{m}\right) W^{[l]} - \alpha d W^{[l]}.$$

Use: L2 regularization as multiplicative weight shrinkage.

9.4 Dropout Mask

Bernoulli mask

$$D^{[l]} \sim \text{Bernoulli}(p_{\text{keep}}), \quad D^{[l]} \in \{0, 1\}^{n^{[l]} \times m}.$$

Use: Dropout mask for layer l .

Notes: Entries of $D^{[l]}$ are sampled elementwise.

9.5 Inverted Dropout Forward Pass

Inverted dropout

$$\begin{aligned} \tilde{A}^{[l]} &= D^{[l]} \odot A^{[l]}. \\ A_{\text{drop}}^{[l]} &= \frac{\tilde{A}^{[l]}}{p_{\text{keep}}} = \frac{D^{[l]} \odot A^{[l]}}{p_{\text{keep}}}. \\ A^{[l]}, D^{[l]}, A_{\text{drop}}^{[l]} &\in \mathbb{R}^{n^{[l]} \times m}. \end{aligned}$$

Use: Inverted dropout during training.

9.6 Dropout Train/Test Convention

Training

$$A^{[l]} \leftarrow \frac{D^{[l]} \odot A^{[l]}}{p_{\text{keep}}}.$$

Use: Training-time inverted dropout scaling.

Test / inference

$$A_{\text{test}}^{[l]} = A^{[l]}.$$

Use: Inference-time activation convention.

9.7 Other Regularization Labels

Method	Formula / Label	Use
Early stopping	stop at $\arg \min_t \mathcal{J}_{\text{dev}}(t)$	Dev-set selection rule
Data augmentation	$(x, y) \mapsto (T(x), y)$	Label-preserving transform

Notes: T is a label-preserving transformation.

9.8 Regularization Symbol Table

Symbol	Meaning	Shape / Type
$\mathcal{J}$	unregularized cost	scalar
$\mathcal{J}_{\text{reg}}$	regularized cost	scalar
λ	regularization strength	$\mathbb{R}_{\geq 0}$
m	batch size	$\mathbb{N}$
L	number of layers	$\mathbb{N}$
$W^{[l]}$	layer weights	$\mathbb{R}^{n^{[l]} \times n^{[l-1]}}$
$\ W^{[l]}\ _F^2$	squared Frobenius norm	scalar
$dW_{\text{reg}}^{[l]}$	regularized weight gradient	$\mathbb{R}^{n^{[l]} \times n^{[l-1]}}$
α	learning rate	scalar
$D^{[l]}$	dropout mask	$\{0, 1\}^{n^{[l]} \times m}$
p_{keep}	keep probability	$(0, 1]$
$A^{[l]}$	activation	$\mathbb{R}^{n^{[l]} \times m}$
$A_{\text{drop}}^{[l]}$	dropout-scaled activation	$\mathbb{R}^{n^{[l]} \times m}$
$\mathcal{J}_{\text{dev}}(t)$	dev cost at step t	scalar
$T(x)$	transformed input	same label as x

10 Batch Normalization

10.1 Mini-batch Statistics

Mini-batch mean

$$\mu_B^{[l]} = \frac{1}{m} \sum_{i=1}^m z^{[l](i)}.$$

Mini-batch variance

$$(\sigma_B^2)^{[l]} = \frac{1}{m} \sum_{i=1}^m \left(z^{[l](i)} - \mu_B^{[l]} \right)^{\odot 2}.$$

Shapes: $\mu_B^{[l]}, (\sigma_B^2)^{[l]} \in \mathbb{R}^{n^{[l]} \times 1}$.

Use: Mini-batch mean and variance.

10.2 Normalize

Single-example normalization

$$\hat{z}^{[l](i)} = \frac{z^{[l](i)} - \mu_B^{[l]}}{\sqrt{(\sigma_B^2)^{[l]} + \epsilon}}.$$

Shapes: $z^{[l](i)}, \hat{z}^{[l](i)} \in \mathbb{R}^{n^{[l]} \times 1}$.

Vectorized normalization

$$\hat{Z}^{[l]} = \frac{Z^{[l]} - \mu_B^{[l]}}{\sqrt{(\sigma_B^2)^{[l]} + \epsilon}}.$$

Shapes: $Z^{[l]}, \hat{Z}^{[l]} \in \mathbb{R}^{n^{[l]} \times m}$.

Use: Normalize dense-layer pre-activations.

Notes: Subtraction and division broadcast across the batch axis. Square root and division are elementwise.

10.3 Scale and Shift

Single-example scale and shift

$$\tilde{z}^{[l](i)} = \gamma^{[l]} \odot \hat{z}^{[l](i)} + \beta^{[l]}.$$

Vectorized scale and shift

$$\begin{aligned}\tilde{Z}^{[l]} &= \gamma^{[l]} \odot \hat{Z}^{[l]} + \beta^{[l]}. \\ \gamma^{[l]}, \beta^{[l]} &\in \mathbb{R}^{n^{[l]} \times 1}, \quad \hat{Z}^{[l]}, \tilde{Z}^{[l]} \in \mathbb{R}^{n^{[l]} \times m}.\end{aligned}$$

Use: Learnable scale and shift after normalization.

10.4 BatchNorm Forward Pipeline

Dense-layer BatchNorm pipeline

$$Z^{[l]} \rightarrow \mu_B^{[l]}, (\sigma_B^2)^{[l]} \rightarrow \hat{Z}^{[l]} \rightarrow \tilde{Z}^{[l]} \rightarrow A^{[l]} = g^{[l]}(\tilde{Z}^{[l]}).$$

Use: Dense-layer BatchNorm forward pass.

10.5 Running Mean and Variance

Running estimates

$$\begin{aligned}\mu_{\text{run}}^{[l]} &\leftarrow \rho \mu_{\text{run}}^{[l]} + (1 - \rho) \mu_B^{[l]}. \\ (\sigma_{\text{run}}^2)^{[l]} &\leftarrow \rho (\sigma_{\text{run}}^2)^{[l]} + (1 - \rho) (\sigma_B^2)^{[l]}.\end{aligned}$$

Shapes: $\mu_{\text{run}}^{[l]}, (\sigma_{\text{run}}^2)^{[l]} \in \mathbb{R}^{n^{[l]} \times 1}$.

Use: Maintain population estimates for inference.

10.6 Training vs Inference

Training normalization

$$\hat{Z}_{\text{train}}^{[l]} = \frac{Z^{[l]} - \mu_B^{[l]}}{\sqrt{(\sigma_B^2)^{[l]} + \epsilon}}.$$

Training scale and shift

$$\tilde{Z}_{\text{train}}^{[l]} = \gamma^{[l]} \odot \hat{Z}_{\text{train}}^{[l]} + \beta^{[l]}.$$

Inference normalization

$$\hat{Z}_{\text{test}}^{[l]} = \frac{Z^{[l]} - \mu_{\text{run}}^{[l]}}{\sqrt{(\sigma_{\text{run}}^2)^{[l]} + \epsilon}}.$$

Inference scale and shift

$$\tilde{Z}_{\text{test}}^{[l]} = \gamma^{[l]} \odot \hat{Z}_{\text{test}}^{[l]} + \beta^{[l]}.$$

Use: Batch statistics during training; running statistics during inference.

Notes: Train/test subscripts have the same shape as $\hat{Z}^{[l]}$ and $\tilde{Z}^{[l]}$.

10.7 Parameter Shapes

Quantity	Shape
$Z^{[l]}$	$\mathbb{R}^{n^{[l]} \times m}$
$z^{[l](i)}$	$\mathbb{R}^{n^{[l]} \times 1}$
$\mu_B^{[l]}$	$\mathbb{R}^{n^{[l]} \times 1}$
$(\sigma_B^2)^{[l]}$	$\mathbb{R}^{n^{[l]} \times 1}$
$\hat{Z}^{[l]}$	$\mathbb{R}^{n^{[l]} \times m}$
$\tilde{Z}^{[l]}$	$\mathbb{R}^{n^{[l]} \times m}$
$\gamma^{[l]}$	$\mathbb{R}^{n^{[l]} \times 1}$
$\beta^{[l]}$	$\mathbb{R}^{n^{[l]} \times 1}$
$\mu_{\text{run}}^{[l]}$	$\mathbb{R}^{n^{[l]} \times 1}$
$(\sigma_{\text{run}}^2)^{[l]}$	$\mathbb{R}^{n^{[l]} \times 1}$

10.8 BatchNorm Symbol Table

Symbol	Meaning	Shape / Type
$Z^{[l]}$	pre-activation batch	$\mathbb{R}^{n^{[l]} \times m}$
$z^{[l](i)}$	single pre-activation column	$\mathbb{R}^{n^{[l]} \times 1}$
$\mu_B^{[l]}$	mini-batch mean	$\mathbb{R}^{n^{[l]} \times 1}$
$(\sigma_B^2)^{[l]}$	mini-batch variance	$\mathbb{R}^{n^{[l]} \times 1}$
$\hat{Z}^{[l]}$	normalized pre-activation	$\mathbb{R}^{n^{[l]} \times m}$
$\tilde{Z}^{[l]}$	scaled and shifted pre-activation	$\mathbb{R}^{n^{[l]} \times m}$
$\gamma^{[l]}$	scale parameter	$\mathbb{R}^{n^{[l]} \times 1}$
$\beta^{[l]}$	shift parameter	$\mathbb{R}^{n^{[l]} \times 1}$
ϵ	denominator stabilizer	small scalar
ρ	running-average coefficient	scalar in $[0, 1]$
$\mu_{\text{run}}^{[l]}$	running mean	$\mathbb{R}^{n^{[l]} \times 1}$
$(\sigma_{\text{run}}^2)^{[l]}$	running variance	$\mathbb{R}^{n^{[l]} \times 1}$
m	batch size	$\mathbb{N}$
$g^{[l]}$	activation function	layer map

11 Convolutional Neural Networks

11.1 CNN Tensor Convention [Derived]

NHWC activations

$$A^{[l-1]} \in \mathbb{R}^{m \times n_H^{[l-1]} \times n_W^{[l-1]} \times n_C^{[l-1]}}, \quad A^{[l]} \in \mathbb{R}^{m \times n_H^{[l]} \times n_W^{[l]} \times n_C^{[l]}}.$$

Use: CNN batch tensor convention.

Notes: $n_C^{[l]}$ equals the number of filters in layer l .

11.2 Convolution Output Size [Core]

Spatial output size

$$n_H^{[l]} = \left\lfloor \frac{n_H^{[l-1]} + 2p^{[l]} - f^{[l]}}{s^{[l]}} \right\rfloor + 1.$$

$$n_W^{[l]} = \left\lfloor \frac{n_W^{[l-1]} + 2p^{[l]} - f^{[l]}}{s^{[l]}} \right\rfloor + 1.$$

$$n_C^{[l]} = \text{number of filters}, \quad A^{[l]} \in \mathbb{R}^{m \times n_H^{[l]} \times n_W^{[l]} \times n_C^{[l]}}.$$

Use: Convolution spatial output size.

11.3 Multi-channel Convolution [Derived]

One output position and channel

$$Z_{h,w,k}^{[l](i)} = \sum_{u=1}^{f^{[l]}} \sum_{v=1}^{f^{[l]}} \sum_{c=1}^{n_C^{[l-1]}} A_{h_u, w_v, c}^{[l-1](i)} W_{u,v,c,k}^{[l]} + b_k^{[l]}.$$

$$W^{[l]} \in \mathbb{R}^{f^{[l]} \times f^{[l]} \times n_C^{[l-1]} \times n_C^{[l]}}, \quad b^{[l]} \in \mathbb{R}^{1 \times 1 \times 1 \times n_C^{[l]}}.$$

Use: Multi-channel convolution for one spatial position and one output channel.

Notes: h_u, w_v denote stride-adjusted input positions inside the filter window.

11.4 Convolution Parameter Count [Derived]

Parameter count

$$\#\text{params}_{\text{per filter}} = f^{[l]} f^{[l]} n_C^{[l-1]} + 1.$$

$$\#\text{params}_{\text{layer}} = \left(f^{[l]} f^{[l]} n_C^{[l-1]} + 1 \right) n_C^{[l]}.$$

Use: Convolution layer parameter count including bias.

11.5 Padding Conventions [Derived]

Convention	Formula	Use
Valid	$p = 0$	no padding
Same, $s = 1$	$p = \frac{f-1}{2}$	preserve spatial size

Notes: Same padding requires suitable integer padding; for odd f , $p = (f-1)/2$.

11.6 Pooling Output Size [Derived]

Pooling spatial output size

$$n_H^{[l]} = \left\lfloor \frac{n_H^{[l-1]} - f}{s} \right\rfloor + 1, \quad n_W^{[l]} = \left\lfloor \frac{n_W^{[l-1]} - f}{s} \right\rfloor + 1.$$

$$n_C^{[l]} = n_C^{[l-1]}.$$

Use: Pooling spatial output size.

11.7 Pooling Operations [Core]

Max pooling

$$A_{h,w,c}^{[l](i)} = \max_{(u,v) \in \mathcal{R}_{h,w}} A_{u,v,c}^{[l-1](i)}.$$

Average pooling

$$A_{h,w,c}^{[l](i)} = \frac{1}{|\mathcal{R}_{h,w}|} \sum_{(u,v) \in \mathcal{R}_{h,w}} A_{u,v,c}^{[l-1](i)}.$$

Use: Pooling over spatial regions.

Notes: $\mathcal{R}_{h,w}$ is the pooling window for output position (h, w) .

11.8 1x1 Convolution [Optional]

Parameter count and channel projection

$$\begin{aligned}\#\text{params} &= (1 \cdot 1 \cdot n_C^{[l-1]} + 1)n_C^{[l]}. \\ A^{[l-1]} &\in \mathbb{R}^{m \times n_H \times n_W \times n_C^{[l-1]}} \rightarrow A^{[l]} \in \mathbb{R}^{m \times n_H \times n_W \times n_C^{[l]}}.\end{aligned}$$

Use: Channel projection / bottleneck convolution.

11.9 Depthwise Separable Convolution [Optional]

Standard convolution parameters

$$\#\text{params}_{\text{standard}} = k^2 C_{\text{in}} C_{\text{out}}.$$

Depthwise separable parameters

$$\#\text{params}_{\text{dwsep}} = k^2 C_{\text{in}} + C_{\text{in}} C_{\text{out}}.$$

Shapes: k is filter size; C_{in} input channels; C_{out} output channels.

Use: Reduces parameters in MobileNet-style convolutions.

Notes: Depthwise plus pointwise (1×1) convolution.

11.10 CNN Symbol Table

Symbol	Meaning	Shape / Type
$A^{[l-1]}$	input activation	$\mathbb{R}^{m \times n_H^{[l-1]} \times n_W^{[l-1]} \times n_C^{[l-1]}}$
$A^{[l]}$	output activation	$\mathbb{R}^{m \times n_H^{[l]} \times n_W^{[l]} \times n_C^{[l]}}$
$Z^{[l]}$	convolution pre-activation	$\mathbb{R}^{m \times n_H^{[l]} \times n_W^{[l]} \times n_C^{[l]}}$
$W^{[l]}$	convolution filters	$\mathbb{R}^{f^{[l]} \times f^{[l]} \times n_C^{[l-1]} \times n_C^{[l]}}$
$b^{[l]}$	convolution bias	$\mathbb{R}^{1 \times 1 \times 1 \times n_C^{[l]}}$
m	batch size	$\mathbb{N}$
$n_H^{[l]}$	height at layer l	$\mathbb{N}$
$n_W^{[l]}$	width at layer l	$\mathbb{N}$
$n_C^{[l]}$	channels / filters at layer l	$\mathbb{N}$
$f^{[l]}$	filter size	$\mathbb{N}$
$p^{[l]}$	padding	$\mathbb{N}_0$
$s^{[l]}$	stride	$\mathbb{N}$
k	depthwise filter size	$\mathbb{N}$
C_{in}	input channels	$\mathbb{N}$
C_{out}	output channels	$\mathbb{N}$
$\mathcal{R}_{h,w}$	pooling window	spatial index set

12 Classic CNN Architectures

12.1 Architecture Table

Architecture	Core pattern	Formula / Shape label
LeNet	Conv / pool stack	Conv $\rightarrow$ Pool $\rightarrow$ Conv $\rightarrow$ Pool $\rightarrow$ FC
AlexNet	Convolution stack with FC head	Conv ^(k) $\rightarrow$ FC
VGG	Repeated small filters	[Conv _{3×3}] $\times k \rightarrow$ Pool
ResNet	Shortcut addition	$A^{[l+2]} = g(Z^{[l+2]} + A^{[l]})$
Inception	Parallel branches	$A^{[l]} = \text{Concat}(A_{1\times 1}, A_{3\times 3}, A_{5\times 5}, A_{\text{pool}})$

12.2 VGG Block Pattern

VGG-style block

$$A^{[l]} = \text{Pool} \left(\text{Conv}_{3\times 3}^{(k)}(A^{[l-1]}) \right).$$

Use: Repeated small-filter convolution block.

12.3 Residual Block

Identity shortcut

$$A^{[l+2]} = g \left(Z^{[l+2]} + A^{[l]} \right).$$

Projection shortcut

$$A^{[l+2]} = g \left(Z^{[l+2]} + W_s A^{[l]} \right).$$

Use: Residual connection.

Notes: For identity shortcuts, $Z^{[l+2]}$ and $A^{[l]}$ must be shape-compatible. For projection shortcuts, $W_s A^{[l]}$ matches the shape of $Z^{[l+2]}$.

12.4 Inception Concatenation

Channel concatenation

$$A^{[l]} = \text{Concat}_{\text{channels}}(A_{1\times 1}, A_{3\times 3}, A_{5\times 5}, A_{\text{pool}}).$$

$$n_C^{[l]} = n_{C,1\times 1} + n_{C,3\times 3} + n_{C,5\times 5} + n_{C,\text{pool}}.$$

Use: Parallel convolution branches with channel concatenation.

12.5 1x1 Bottleneck Parameter Reduction

Without bottleneck

$$\# \text{params} = (f^2 n_C^{\text{in}} + 1) n_C^{\text{out}}.$$

With 1x1 bottleneck width r

$$\# \text{params} = (1 \cdot 1 \cdot n_C^{\text{in}} + 1) r + (f^2 r + 1) n_C^{\text{out}}.$$

Use: Parameter reduction before expensive convolution.

12.6 Architecture Symbol Table

Symbol	Meaning	Shape / Type
$A^{[l]}$	activation at layer l	layer tensor
$Z^{[l]}$	pre-activation at layer l	layer tensor
g	activation function	layer map
W_s	shortcut projection	shape-matching map
$n_C^{[l]}$	output channels at layer l	$\mathbb{N}$
r	bottleneck width	$\mathbb{N}$
$\text{Concat}_{\text{channels}}$	channel-axis concatenation	NHWC channel axis

13 Object Detection and YOLO

13.1 Bounding Box Representation

Box parameterizations

$$b = (b_x, b_y, b_h, b_w), \quad b = (x_{\min}, y_{\min}, x_{\max}, y_{\max}).$$

Use: Bounding box coordinates.

Notes: Use one representation consistently within a formula.

13.2 Intersection over Union

IoU

$$\text{IoU}(B_1, B_2) = \frac{\text{area}(B_1 \cap B_2)}{\text{area}(B_1 \cup B_2)}.$$

Use: Bounding-box overlap score.

13.3 YOLO Grid Output Shape

Course-style YOLO output

$$S \times S \times (B \cdot 5 + C).$$

$$\text{Batch output} \in \mathbb{R}^{m \times S \times S \times (B \cdot 5 + C)}.$$

$$(p_c, b_x, b_y, b_h, b_w).$$

Use: YOLO grid-cell prediction tensor.

Notes: Modern YOLO variants may use different head designs; deferred to V2.

13.4 Objectness and Class Scores

Class-specific confidence

$$p_c = P(\text{object}), \quad P(c_j \mid \text{object}).$$

$$\text{score}_j = p_c P(c_j \mid \text{object}).$$

Use: Class confidence score for detection.

13.5 Anchor Boxes

Anchor assignment

$$\text{anchor} = \arg \max_a \text{IoU}(B_{\text{true}}, A_a).$$

Use: Assign ground-truth box to best matching anchor.

13.6 Non-max Suppression

Selection and suppression

$$B^* = \arg \max_{B_i} \text{score}(B_i).$$

$$\text{IoU}(B_i, B^*) > \tau \quad \Rightarrow \quad \text{discard } B_i.$$

Use: Remove duplicate boxes.

13.7 Object Detection Symbol Table

Symbol	Meaning	Shape / Type
S	grid size	$\mathbb{N}$
B	boxes per grid cell	$\mathbb{N}$
C	object classes	$\mathbb{N}$
p_c	objectness probability	scalar
b_x, b_y, b_h, b_w	box center/size coordinates	scalars
B_i	predicted box	box
A_a	anchor box a	box
IoU	intersection over union	$[0, 1]$
τ	NMS threshold	scalar
score_j	class confidence score	scalar

14 Face Recognition and Neural Style Transfer

14.1 Embedding Function

Image embedding

$$f(x) \in \mathbb{R}^d.$$

Use: Map input image to embedding vector.

14.2 Pairwise Distance

Embedding-space distance

$$d(x_i, x_j) = \|f(x_i) - f(x_j)\|_2^2.$$

Use: Embedding-space distance.

14.3 Siamese / Triplet Loss

Triplet

$$(A, P, N).$$

Use: Anchor, positive, negative.

Triplet loss

$$\mathcal{L}_{\text{triplet}} = \max \left(\|f(A) - f(P)\|_2^2 - \|f(A) - f(N)\|_2^2 + \alpha, 0 \right).$$

Use: Separate positive and negative pairs by margin α .

14.4 Content Cost

Simplified content cost

$$\mathcal{J}_{\text{content}}(C, G) = \frac{1}{2} \left\| A_C^{[l]} - A_G^{[l]} \right\|_F^2.$$

Use: Content similarity between generated and content image activations.

Course-normalized content cost

$$\mathcal{J}_{\text{content}}(C, G) = \frac{1}{4n_H^{[l]}n_W^{[l]}n_C^{[l]}} \left\| A_C^{[l]} - A_G^{[l]} \right\|_F^2.$$

Use: Course-normalized content objective.

14.5 Gram Matrix

Style Gram matrix

$$F^{[l]} \in \mathbb{R}^{n_C^{[l]} \times (n_H^{[l]} n_W^{[l]})}, \quad G^{[l]} = F^{[l]} (F^{[l]})^\top.$$
$$G^{[l]} \in \mathbb{R}^{n_C^{[l]} \times n_C^{[l]}}.$$

Use: Channel-correlation matrix for style.

14.6 Style Cost

Layer style cost

$$\mathcal{J}_{\text{style}}^{[l]}(S, G) = \frac{1}{(2n_H^{[l]}n_W^{[l]}n_C^{[l]})^2} \left\| G_S^{[l]} - G_G^{[l]} \right\|_F^2.$$

Total style cost

$$\mathcal{J}_{\text{style}}(S, G) = \sum_l \lambda_l \mathcal{J}_{\text{style}}^{[l]}(S, G).$$

Use: Style similarity across selected layers.

14.7 Total Neural Style Transfer Objective

Style transfer objective

$$\mathcal{J}(G) = \alpha \mathcal{J}_{\text{content}}(C, G) + \beta \mathcal{J}_{\text{style}}(S, G).$$

Use: Neural style transfer objective.

Notes: Here α, β are style-transfer weights, not optimizer learning rate or BatchNorm shift.

14.8 Symbol Table

Symbol	Meaning	Shape / Type
$f(x)$	image embedding	$\mathbb{R}^d$
d	embedding dimension	$\mathbb{N}$
A, P, N	anchor, positive, negative images	images
$\mathcal{L}_{\text{triplet}}$	triplet loss	scalar
α in triplet loss	margin	scalar
C, S, G	content, style, generated images	images
$A_C^{[l]}$	content activation at layer l	layer tensor
$A_G^{[l]}$	generated activation at layer l	layer tensor
$F^{[l]}$	reshaped style features	$\mathbb{R}^{n_C^{[l]} \times (n_H^{[l]} n_W^{[l]})}$
$G^{[l]}$	Gram matrix	$\mathbb{R}^{n_C^{[l]} \times n_C^{[l]}}$
$G_S^{[l]}$	style-image Gram matrix	$\mathbb{R}^{n_C^{[l]} \times n_C^{[l]}}$
$G_G^{[l]}$	generated-image Gram matrix	$\mathbb{R}^{n_C^{[l]} \times n_C^{[l]}}$
$\mathcal{J}_{\text{content}}$	content cost	scalar
$\mathcal{J}_{\text{style}}$	style cost	scalar
λ_l	style-layer weight	scalar
α, β in style transfer	objective weights	scalars

Notes: α in triplet loss is a margin. α, β in style transfer are objective weights. These are not the optimizer learning rate unless explicitly stated.

15 RNN / GRU / LSTM

15.1 Sequence Tensor Convention

Sequence tensors

$$x^{(t)} \in \mathbb{R}^{n_x \times m}, \quad a^{(t)} \in \mathbb{R}^{n_a \times m}, \quad \hat{y}^{(t)} \in \mathbb{R}^{n_y \times m}.$$

$$c^{(t)} \in \mathbb{R}^{n_a \times m}.$$

$$t = 1, \dots, T_x \quad \text{for input sequence steps.}$$

$$t = 1, \dots, T_y \quad \text{for output sequence steps.}$$

Use: Sequence-model tensor convention.

Notes: $a^{(0)}$ and $c^{(0)}$ are initial states.

15.2 Vanilla RNN Forward Pass

Concatenated input

$$\begin{bmatrix} a^{(t-1)} \\ x^{(t)} \end{bmatrix} \in \mathbb{R}^{(n_a + n_x) \times m}.$$

Use: RNN state/input stack.

Hidden-state update

$$a^{(t)} = \tanh \left(W_a \begin{bmatrix} a^{(t-1)} \\ x^{(t)} \end{bmatrix} + b_a \right).$$

$$W_a \in \mathbb{R}^{n_a \times (n_a + n_x)}, \quad b_a \in \mathbb{R}^{n_a \times 1}, \quad a^{(t)} \in \mathbb{R}^{n_a \times m}.$$

Use: Vanilla RNN hidden-state update.

Equivalent split form

$$a^{(t)} = \tanh\left(W_{aa}a^{(t-1)} + W_{ax}x^{(t)} + b_a\right).$$

$$W_{aa} \in \mathbb{R}^{n_a \times n_a}, \quad W_{ax} \in \mathbb{R}^{n_a \times n_x}.$$

Use: Split recurrent/input weights.

15.3 RNN Output Prediction

Output layer

$$z_y^{(t)} = W_y a^{(t)} + b_y.$$

$$\hat{y}^{(t)} = g_y\left(z_y^{(t)}\right).$$

$$g_y = \text{softmax} \quad \text{or} \quad g_y = \sigma.$$

$$W_y \in \mathbb{R}^{n_y \times n_a}, \quad b_y \in \mathbb{R}^{n_y \times 1}, \quad z_y^{(t)}, \hat{y}^{(t)} \in \mathbb{R}^{n_y \times m}.$$

Use: RNN output layer.

15.4 Sequence Loss

Sequence loss

$$\mathcal{L} = \sum_{\tau=1}^{T_y} \mathcal{L}^{(\tau)}.$$

Categorical sequence loss

$$\mathcal{L} = - \sum_{\tau=1}^{T_y} \sum_{k=1}^{n_y} y_k^{(\tau)} \log \hat{y}_k^{(\tau)}.$$

Batch average

$$\mathcal{J} = \frac{1}{m} \sum_{i=1}^m \sum_{\tau=1}^{T_y} \mathcal{L}^{(\tau)(i)}.$$

Use: Sequence prediction objective.

15.5 Backpropagation Through Time Label

Time-summed parameter gradient

$$\frac{\partial \mathcal{J}}{\partial W_a} = \sum_{\tau=1}^{T_x} \frac{\partial \mathcal{J}}{\partial W_a} \Big|_{\langle \tau \rangle}.$$

Future-state dependency

$$a^{(t)} \rightarrow a^{(t+1)} \rightarrow \dots \rightarrow \mathcal{J}.$$

Use: BPTT gradient accumulation over time.

15.6 GRU Forward Pass

GRU state

$$c^{(t)} \in \mathbb{R}^{n_a \times m}.$$

Use: GRU memory / hidden state.

GRU gates

$$\begin{aligned}\Gamma_u^{(t)} &= \sigma \left(W_u \begin{bmatrix} c^{(t-1)} \\ x^{(t)} \end{bmatrix} + b_u \right). \\ \Gamma_r^{(t)} &= \sigma \left(W_r \begin{bmatrix} c^{(t-1)} \\ x^{(t)} \end{bmatrix} + b_r \right).\end{aligned}$$

Candidate memory

$$\tilde{c}^{(t)} = \tanh \left(W_{\tilde{c}} \begin{bmatrix} \Gamma_r^{(t)} \odot c^{(t-1)} \\ x^{(t)} \end{bmatrix} + b_{\tilde{c}} \right).$$

Memory update

$$c^{(t)} = \Gamma_u^{(t)} \odot \tilde{c}^{(t)} + (1 - \Gamma_u^{(t)}) \odot c^{(t-1)}.$$

$$\Gamma_u^{(t)}, \Gamma_r^{(t)}, \tilde{c}^{(t)}, c^{(t)} \in \mathbb{R}^{n_a \times m}.$$

$$W_u, W_r, W_{\tilde{c}} \in \mathbb{R}^{n_a \times (n_a + n_x)}, \quad b_u, b_r, b_{\tilde{c}} \in \mathbb{R}^{n_a \times 1}.$$

Use: GRU gated memory update.

Notes: In the GRU block, $W_{\tilde{c}}$ denotes GRU candidate-state weights.

Alternative update-gate convention

Some sources define Γ_u with the opposite convention:

$$c^{(t)} = \Gamma_u^{(t)} \odot c^{(t-1)} + (1 - \Gamma_u^{(t)}) \odot \tilde{c}^{(t)}.$$

15.7 LSTM Forward Pass

LSTM concatenated input

$$\begin{bmatrix} a^{(t-1)} \\ x^{(t)} \end{bmatrix} \in \mathbb{R}^{(n_a + n_x) \times m}.$$

LSTM gates

$$\begin{aligned}\Gamma_f^{(t)} &= \sigma \left(W_f \begin{bmatrix} a^{(t-1)} \\ x^{(t)} \end{bmatrix} + b_f \right). \\ \Gamma_i^{(t)} &= \sigma \left(W_i \begin{bmatrix} a^{(t-1)} \\ x^{(t)} \end{bmatrix} + b_i \right). \\ \tilde{c}^{(t)} &= \tanh \left(W_{\tilde{c}} \begin{bmatrix} a^{(t-1)} \\ x^{(t)} \end{bmatrix} + b_{\tilde{c}} \right). \\ c^{(t)} &= \Gamma_f^{(t)} \odot c^{(t-1)} + \Gamma_i^{(t)} \odot \tilde{c}^{(t)}. \\ \Gamma_o^{(t)} &= \sigma \left(W_o \begin{bmatrix} a^{(t-1)} \\ x^{(t)} \end{bmatrix} + b_o \right). \\ a^{(t)} &= \Gamma_o^{(t)} \odot \tanh(c^{(t)}).\end{aligned}$$

$$\Gamma_f^{(t)}, \Gamma_i^{(t)}, \Gamma_o^{(t)}, \tilde{c}^{(t)}, c^{(t)}, a^{(t)} \in \mathbb{R}^{n_a \times m}.$$

$$W_f, W_i, W_{\tilde{c}}, W_o \in \mathbb{R}^{n_a \times (n_a + n_x)}, \quad b_f, b_i, b_{\tilde{c}}, b_o \in \mathbb{R}^{n_a \times 1}.$$

Use: LSTM gated cell-state update.

Notes: In the LSTM block, $W_{\tilde{c}}$ denotes LSTM candidate-cell weights.

15.8 Bidirectional RNN

Forward and backward states

$$\vec{a}^{(t)} = \text{RNN}_{\rightarrow} \left(x^{(t)}, \vec{a}^{(t-1)} \right).$$

$$\overleftarrow{a}^{(t)} = \text{RNN}_{\leftarrow} \left(x^{(t)}, \overleftarrow{a}^{(t+1)} \right).$$

Concatenated bidirectional state

$$a_{\text{bi}}^{(t)} = \begin{bmatrix} \vec{a}^{(t)} \\ \overleftarrow{a}^{(t)} \end{bmatrix} \in \mathbb{R}^{2n_a \times m}.$$

Use: Bidirectional sequence representation.

15.9 RNN / GRU / LSTM Symbol Table

Symbol	Meaning	Shape / Type
$x^{(t)}$	input at step $\langle t \rangle$	$\mathbb{R}^{n_x \times m}$
$a^{(t)}$	hidden activation	$\mathbb{R}^{n_a \times m}$
$a^{(0)}$	initial hidden activation	$\mathbb{R}^{n_a \times m}$
$c^{(t)}$	cell / GRU state	$\mathbb{R}^{n_a \times m}$
$c^{(0)}$	initial cell / GRU state	$\mathbb{R}^{n_a \times m}$
$\hat{y}^{(t)}$	output prediction	$\mathbb{R}^{n_y \times m}$
$z_y^{(t)}$	output logits	$\mathbb{R}^{n_y \times m}$
W_a	RNN hidden weights	$\mathbb{R}^{n_a \times (n_a + n_x)}$
b_a	RNN hidden bias	$\mathbb{R}^{n_a \times 1}$
W_y	output weights	$\mathbb{R}^{n_y \times n_a}$
b_y	output bias	$\mathbb{R}^{n_y \times 1}$
W_u	GRU update-gate weights	$\mathbb{R}^{n_a \times (n_a + n_x)}$
b_u	GRU update-gate bias	$\mathbb{R}^{n_a \times 1}$
W_r	GRU reset-gate weights	$\mathbb{R}^{n_a \times (n_a + n_x)}$
b_r	GRU reset-gate bias	$\mathbb{R}^{n_a \times 1}$
$W_{\tilde{c}}$	candidate-state weights	$\mathbb{R}^{n_a \times (n_a + n_x)}$
$b_{\tilde{c}}$	candidate-state bias	$\mathbb{R}^{n_a \times 1}$
W_f	LSTM forget-gate weights	$\mathbb{R}^{n_a \times (n_a + n_x)}$
b_f	LSTM forget-gate bias	$\mathbb{R}^{n_a \times 1}$
W_i	LSTM input-gate weights	$\mathbb{R}^{n_a \times (n_a + n_x)}$
b_i	LSTM input-gate bias	$\mathbb{R}^{n_a \times 1}$
W_o	LSTM output-gate weights	$\mathbb{R}^{n_a \times (n_a + n_x)}$
b_o	LSTM output-gate bias	$\mathbb{R}^{n_a \times 1}$
$\Gamma_u^{(t)}$	GRU update gate	$\mathbb{R}^{n_a \times m}$
$\Gamma_r^{(t)}$	GRU reset gate	$\mathbb{R}^{n_a \times m}$
$\Gamma_f^{(t)}$	LSTM forget gate	$\mathbb{R}^{n_a \times m}$
$\Gamma_i^{(t)}$	LSTM input gate	$\mathbb{R}^{n_a \times m}$
$\Gamma_o^{(t)}$	LSTM output gate	$\mathbb{R}^{n_a \times m}$
$\tilde{c}^{(t)}$	candidate state	$\mathbb{R}^{n_a \times m}$
T_x	input sequence length	$\mathbb{N}$
T_y	output sequence length	$\mathbb{N}$
m	batch size	$\mathbb{N}$
n_x	input dimension	$\mathbb{N}$
n_a	hidden-state dimension	$\mathbb{N}$
n_y	output dimension	$\mathbb{N}$

15.10 Shape Reference Table

Quantity	Shape
$x^{(t)}$	$n_x \times m$
$a^{(t)}$	$n_a \times m$
$c^{(t)}$	$n_a \times m$
$\hat{y}^{(t)}$	$n_y \times m$
W_a	$n_a \times (n_a + n_x)$
b_a	$n_a \times 1$
W_y	$n_y \times n_a$
b_y	$n_y \times 1$
$W_u, W_r, W_{\tilde{c}}$ (GRU)	$n_a \times (n_a + n_x)$
$W_f, W_i, W_{\tilde{c}}, W_o$ (LSTM)	$n_a \times (n_a + n_x)$
$a_{\text{bi}}^{(t)}$	$2n_a \times m$

Notes: $W_{\tilde{c}}$ is reused as a candidate-weight symbol; local GRU/LSTM context determines which parameter set is meant.

16 Word Embeddings and Language Models

16.1 Vocabulary and One-hot Representation [Derived]

One-hot token

$$o_w \in \{0, 1\}^{|V| \times 1}.$$

Batch of one-hot tokens

$$O^{(t)} \in \{0, 1\}^{|V| \times m}.$$

Use: One-hot token representation.

16.2 Embedding Matrix and Lookup [Core]

Embedding matrix

$$E \in \mathbb{R}^{d_e \times |V|}.$$

Single-token lookup

$$e_w = E o_w.$$

$$e_w \in \mathbb{R}^{d_e \times 1}.$$

Batch lookup

$$E^{(t)} = E O^{(t)}.$$

$$E^{(t)} \in \mathbb{R}^{d_e \times m}.$$

Use: Map one-hot tokens to dense embedding vectors.

Alternative row-wise convention

$$E_{\text{row}} \in \mathbb{R}^{|V| \times d_e} \Rightarrow e_w = E_{\text{row}}^T o_w.$$

Use: Alternative row-wise embedding convention.

16.3 Embedding Sequence Tensor [Derived]

Time-step embedding sequence

$$E^{(1)}, \dots, E^{(T_x)}, \quad E^{(t)} \in \mathbb{R}^{d_e \times m}.$$

Optional stacked form

$$\mathcal{E} \in \mathbb{R}^{T_x \times m \times d_e}.$$

Use: Sequence of embedding vectors.

Notes: Main convention keeps time steps as separate $d_e \times m$ matrices; frameworks may stack batch-first or time-first.

16.4 Cosine Similarity [Core]

Cosine similarity

$$\cos(u, v) = \frac{u^\top v}{\|u\|_2 \|v\|_2}, \quad u, v \in \mathbb{R}^{d_e}.$$

Use: Embedding similarity.

16.5 Word Analogy Objective [Optional]

Analogy vector

$$v_b - v_a + v_c.$$

Nearest word

$$d^* = \arg \max_{w \in V} \cos(v_w, v_b - v_a + v_c).$$

Use: Vector-offset analogy search.

Notes: Exclude source words from the candidate set if needed.

16.6 Language Model Softmax [Core]

Vocabulary logits

$$\begin{aligned} a^{(t)} &\in \mathbb{R}^{n_a \times m}. \\ z^{(t)} &= W_o a^{(t)} + b_o. \end{aligned}$$

Softmax prediction

$$\begin{aligned} \hat{y}^{(t)} &= \text{softmax}(z^{(t)}). \\ W_o &\in \mathbb{R}^{|V| \times n_a}, \quad b_o \in \mathbb{R}^{|V| \times 1}. \\ z^{(t)}, \hat{y}^{(t)} &\in \mathbb{R}^{|V| \times m}. \end{aligned}$$

Use: Next-token probability distribution.

16.7 Vocabulary Cross-Entropy [Core]

Single time-step loss

$$\mathcal{L}^{(t)} = - \sum_{k=1}^{|V|} y_k^{(t)} \log \hat{y}_k^{(t)}.$$

Sequence loss

$$\mathcal{L} = - \sum_{\tau=1}^{T_y} \sum_{k=1}^{|V|} y_k^{(\tau)} \log \hat{y}_k^{(\tau)}.$$

Batch average

$$\mathcal{J} = \frac{1}{m} \sum_{i=1}^m \mathcal{L}^{(i)}.$$

Use: Language-model negative log-likelihood.

16.8 Perplexity [Derived]

Average token negative log-likelihood

$$\bar{\mathcal{L}} = -\frac{1}{N} \sum_{i=1}^N \log P(w_i | w_{<i}).$$

Perplexity

$$\text{PPL} = \exp(\bar{\mathcal{L}}).$$

Use: Language-model evaluation metric.

Notes: N is the number of evaluated tokens; $w_{<i}$ denotes tokens before position i .

16.9 Skip-gram Objective [Optional]

Skip-gram log-likelihood

$$\max \sum \log P(w_{\text{context}} | w_{\text{center}}).$$

Use: Predict context words from a center word.

Softmax context probability

$$P(w_o | w_c) = \frac{\exp(u_o^T v_c)}{\sum_{w \in V} \exp(u_w^T v_c)}.$$

Shapes: $u_o, u_w, v_c \in \mathbb{R}^{d_e}$.

Use: Skip-gram softmax objective.

16.10 Negative Sampling [Core]

Negative-sampling objective

$$\log \sigma(u_o^T v_c) + \sum_{k=1}^K \mathbb{E}_{w_k \sim P_n} [\log \sigma(-u_k^T v_c)].$$

Shapes: $u_o, u_k, v_c \in \mathbb{R}^{d_e}$; K is the number of negative samples.

Use: Efficient approximation to softmax.

16.11 GloVe Objective [Derived]

Weighted co-occurrence objective

$$\mathcal{J} = \sum_{i,j} f(X_{ij}) (w_i^\top w_j + b_i + b_j - \log X_{ij})^2.$$

Shapes: $w_i, w_j \in \mathbb{R}^{d_e}$; X_{ij} is a co-occurrence count.

Use: Uses global co-occurrence statistics.

16.12 Word Embedding Symbol Table

Symbol	Meaning	Shape / Type
$ V $	vocabulary size	$\mathbb{N}$
d_e	embedding dimension	$\mathbb{N}$
w	token / word	element of V
o_w	one-hot vector for token w	$\{0, 1\}^{ V \times 1}$
$O^{(t)}$	one-hot batch at step $\langle t \rangle$	$\{0, 1\}^{ V \times m}$
E	column-wise embedding matrix	$\mathbb{R}^{d_e \times V }$
E_{row}	optional row-wise embedding matrix	$\mathbb{R}^{ V \times d_e}$
e_w	token embedding	$\mathbb{R}^{d_e \times 1}$
$E^{(t)}$	embedding batch at step $\langle t \rangle$	$\mathbb{R}^{d_e \times m}$
v_w	embedding vector for word w	$\mathbb{R}^{d_e}$
$a^{(t)}$	language-model hidden state	$\mathbb{R}^{n_a \times m}$
W_o	vocabulary output weights	$\mathbb{R}^{ V \times n_a}$
b_o	vocabulary output bias	$\mathbb{R}^{ V \times 1}$
$z^{(t)}$	vocabulary logits	$\mathbb{R}^{ V \times m}$
$\hat{y}^{(t)}$	predicted token distribution	$\mathbb{R}^{ V \times m}$
$y^{(t)}$	target token distribution	$\mathbb{R}^{ V \times m}$
T_x	input sequence length	$\mathbb{N}$
T_y	output sequence length	$\mathbb{N}$
N	evaluated token count	$\mathbb{N}$
PPL	perplexity	scalar
w_c, w_o	center / output word	elements of V
u_w, u_o, u_k	output embedding vectors	$\mathbb{R}^{d_e}$
v_c	center embedding vector	$\mathbb{R}^{d_e}$
K	negative-sample count	$\mathbb{N}$
P_n	negative-sampling distribution	distribution over V
X_{ij}	co-occurrence count	nonnegative scalar
$f(X_{ij})$	GloVe weighting function	scalar weight
b_i, b_j	GloVe biases	scalars

16.13 Shape Reference Table

Quantity	Shape
o_w	$ V \times 1$
$O^{(t)}$	$ V \times m$
E	$d_e \times V $
e_w	$d_e \times 1$
$E^{(t)}$	$d_e \times m$
$a^{(t)}$	$n_a \times m$
W_o	$ V \times n_a$
b_o	$ V \times 1$
$z^{(t)}, \hat{y}^{(t)}$	$ V \times m$

17 Seq2Seq, Beam Search, and Attention

17.1 Conditional Sequence Probability [Core]

Autoregressive probability

$$P(y|x) = \prod_{\tau=1}^{T_y} P\left(y^{(\tau)} \mid y^{(1:\tau-1)}, x\right).$$

Log probability

$$\log P(y|x) = \sum_{\tau=1}^{T_y} \log P\left(y^{(\tau)} \mid y^{(1:\tau-1)}, x\right).$$

Use: Autoregressive sequence probability.

17.2 Encoder-Decoder State Notation [Derived]

Encoder states

$$a^{(1)}, \dots, a^{(T_x)}, \quad a^{(t')} \in \mathbb{R}^{n_a \times m}.$$

Decoder state and output

$$\begin{aligned} s^{(t)} &\in \mathbb{R}^{n_s \times m}. \\ \hat{y}^{(t)} &= \text{softmax}\left(W_y s^{(t)} + b_y\right). \\ \hat{y}^{(t)} &\in \mathbb{R}^{n_y \times m}. \end{aligned}$$

Use: Encoder-decoder state notation.

17.3 Greedy Decoding [Derived]

Greedy token choice

$$\hat{w}^{(t)} = \arg \max_{w \in V} P\left(w \mid \hat{w}^{(1:t-1)}, x\right).$$

Use: Greedy token selection.

17.4 Beam Search Score [Core]

Candidate sequence score

$$\text{score}(y) = \sum_{\tau=1}^{T_y} \log P \left(y^{\langle \tau \rangle} \mid y^{\langle 1:\tau-1 \rangle}, x \right).$$

Beam selection

$$\mathcal{B}^{\langle t \rangle} = \text{TopK}_{y^{\langle 1:t \rangle}} \text{score} \left(y^{\langle 1:t \rangle} \right).$$

Use: Keep top K partial sequences.

17.5 Length Normalization [Optional]

Length-normalized score

$$\text{score}_{\text{norm}}(y) = \frac{1}{T_y^\alpha} \sum_{\tau=1}^{T_y} \log P \left(y^{\langle \tau \rangle} \mid y^{\langle 1:\tau-1 \rangle}, x \right).$$

Use: Beam-search score with length penalty.

Notes: α here is a length-normalization exponent, not the optimizer learning rate.

17.6 Attention Energy [Derived]

Additive attention energy

$$e^{\langle t, t' \rangle} = v_a^\top \tanh \left(W_s s^{\langle t-1 \rangle} + W_a a^{\langle t' \rangle} + b_a \right).$$

$$s^{\langle t-1 \rangle} \in \mathbb{R}^{n_s \times m}, \quad a^{\langle t' \rangle} \in \mathbb{R}^{n_a \times m}.$$

$$e^{\langle t, t' \rangle} \in \mathbb{R}^{1 \times m}.$$

$$W_s \in \mathbb{R}^{n_e \times n_s}, \quad W_a \in \mathbb{R}^{n_e \times n_a}, \quad b_a \in \mathbb{R}^{n_e \times 1}, \quad v_a \in \mathbb{R}^{n_e \times 1}.$$

Use: Alignment score between decoder step $\langle t \rangle$ and encoder step $\langle t' \rangle$.

17.7 Attention Weights [Core]

Attention distribution

$$\alpha^{\langle t, t' \rangle} = \frac{\exp(e^{\langle t, t' \rangle})}{\sum_{j=1}^{T_x} \exp(e^{\langle t, j \rangle})}.$$

$$\sum_{t'=1}^{T_x} \alpha^{\langle t, t' \rangle} = 1.$$

Use: Attention distribution over encoder time steps.

Notes: $\alpha^{\langle t, t' \rangle}$ is an attention weight, not the optimizer learning rate.

17.8 Context Vector [Derived]

Attention context

$$c^{\langle t \rangle} = \sum_{t'=1}^{T_x} \alpha^{\langle t, t' \rangle} a^{\langle t' \rangle}.$$

$$c^{\langle t \rangle} \in \mathbb{R}^{n_a \times m}.$$

$$\alpha^{\langle t, t' \rangle} \in \mathbb{R}^{1 \times m}, \quad a^{\langle t' \rangle} \in \mathbb{R}^{n_a \times m}.$$

$$(1 \times m) \odot (n_a \times m) \rightarrow n_a \times m.$$

Use: Weighted encoder summary for decoder step $\langle t \rangle$.

17.9 Attention Decoder Step [Core]

Decoder update

$$s^{(t)} = \text{RNN}_{\text{dec}} \left(s^{(t-1)}, c^{(t)}, y^{(t-1)} \right).$$

Decoder output

$$\hat{y}^{(t)} = \text{softmax} \left(W_y s^{(t)} + b_y \right).$$
$$W_y \in \mathbb{R}^{n_y \times n_s}, \quad b_y \in \mathbb{R}^{n_y \times 1}, \quad \hat{y}^{(t)} \in \mathbb{R}^{n_y \times m}.$$

Context-output variant

$$\hat{y}^{(t)} = \text{softmax} \left(W_o \begin{bmatrix} s^{(t)} \\ c^{(t)} \end{bmatrix} + b_o \right).$$
$$\begin{bmatrix} s^{(t)} \\ c^{(t)} \end{bmatrix} \in \mathbb{R}^{(n_s + n_a) \times m}.$$
$$W_o \in \mathbb{R}^{n_y \times (n_s + n_a)}, \quad b_o \in \mathbb{R}^{n_y \times 1}.$$

Use: Attention-based decoder step.

17.10 BLEU Score [Optional]

BLEU

$$\text{BLEU} = \text{BP} \cdot \exp \left(\sum_{n=1}^N w_n \log p_n \right).$$

BP = brevity penalty.

Shapes: p_n is modified n -gram precision; $\sum_{n=1}^N w_n = 1$.

Use: Sequence-level evaluation metric for translation.

17.11 Seq2Seq / Attention Symbol Table

Symbol	Meaning	Shape / Type
x	input sequence	length T_x
y	output sequence	length T_y
T_x	input sequence length	$\mathbb{N}$
T_y	output sequence length	$\mathbb{N}$
$a^{(t')}$	encoder hidden state	$\mathbb{R}^{n_a \times m}$
$s^{(t)}$	decoder hidden state	$\mathbb{R}^{n_s \times m}$
$y^{(t)}$	output token / target	token or one-hot vector
$\hat{w}^{(t)}$	predicted token at step $\langle t \rangle$	token in V
$\hat{g}^{(t)}$	predicted output distribution	$\mathbb{R}^{n_y \times m}$
K	beam width	$\mathbb{N}$
$\mathcal{B}^{(t)}$	beam set at step $\langle t \rangle$	set of K partial sequences
$\text{score}(y)$	sequence log score	scalar
$\text{score}_{\text{norm}}(y)$	length-normalized score	scalar
α	length-normalization exponent	scalar
$e^{(t,t')}$	attention energy	$\mathbb{R}^{1 \times m}$
$\alpha^{(t,t')}$	attention weight	$\mathbb{R}^{1 \times m}$
$c^{(t)}$	context vector	$\mathbb{R}^{n_a \times m}$
W_s	decoder-state attention weights	$\mathbb{R}^{n_e \times n_s}$
W_a	encoder-state attention weights	$\mathbb{R}^{n_e \times n_a}$
v_a	attention scoring vector	$\mathbb{R}^{n_e \times 1}$
W_y	decoder output weights	$\mathbb{R}^{n_y \times n_s}$
b_y	decoder output bias	$\mathbb{R}^{n_y \times 1}$
W_o	context-output weights	$\mathbb{R}^{n_y \times (n_s + n_a)}$
b_o	context-output bias	$\mathbb{R}^{n_y \times 1}$
n_s	decoder hidden-state dimension	$\mathbb{N}$
n_a	encoder hidden-state dimension	$\mathbb{N}$
n_e	attention hidden dimension	$\mathbb{N}$
BLEU	translation metric	scalar
BP	brevity penalty	scalar
p_n	modified n -gram precision	scalar
w_n	BLEU n -gram weight	scalar
N	maximum n -gram order	$\mathbb{N}$

Notes: α is a length exponent; $\alpha^{(t,t')}$ is an attention weight; neither is the optimizer learning rate unless stated.

17.12 Shape Reference Table

Quantity	Shape
$a^{(t')}$	$n_a \times m$
$s^{(t)}$	$n_s \times m$
$e^{(t,t')}$	$1 \times m$
$\alpha^{(t,t')}$	$1 \times m$
$c^{(t)}$	$n_a \times m$
W_s	$n_e \times n_s$
W_a	$n_e \times n_a$
v_a	$n_e \times 1$
W_y	$n_y \times n_s$
b_y	$n_y \times 1$
W_o	$n_y \times (n_s + n_a)$
b_o	$n_y \times 1$
$\begin{bmatrix} s^{(t)} \\ c^{(t)} \end{bmatrix}$	$(n_s + n_a) \times m$
$\hat{y}^{(t)}$	$n_y \times m$

18 Transformer and Self-Attention

18.1 Token Representation and Q/K/V Projections [Derived]

Input sequence representation

$$X \in \mathbb{R}^{T \times d_{\text{model}}}.$$

Projection matrices

$$W_Q \in \mathbb{R}^{d_{\text{model}} \times d_k}, \quad W_K \in \mathbb{R}^{d_{\text{model}} \times d_k}, \quad W_V \in \mathbb{R}^{d_{\text{model}} \times d_v}.$$

Single-head projections

$$Q = XW_Q, \quad K = XW_K, \quad V = XW_V.$$

$$Q, K \in \mathbb{R}^{T \times d_k}, \quad V \in \mathbb{R}^{T \times d_v}.$$

Cross-attention shapes

$$Q \in \mathbb{R}^{T_q \times d_k}, \quad K \in \mathbb{R}^{T_k \times d_k}, \quad V \in \mathbb{R}^{T_k \times d_v}.$$

Use: Linear projections for attention.

18.2 Scaled Dot-Product Attention [Core]

Attention scores

$$S = \frac{QK^T}{\sqrt{d_k}}.$$

Attention weights

$$P = \text{softmax}(S).$$

$$P_{i,:} = \text{softmax}(S_{i,:}).$$

Notes: Softmax is applied row-wise over key positions.

Attention output

$$O = PV.$$

Full attention formula

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right)V.$$

Use: Scaled dot-product attention.

18.3 Attention Shape Check [Derived]

Score shape

$$QK^\top : (T_q \times d_k)(d_k \times T_k) \rightarrow T_q \times T_k.$$

$$S, P \in \mathbb{R}^{T_q \times T_k}.$$

Output shape

$$PV : (T_q \times T_k)(T_k \times d_v) \rightarrow T_q \times d_v.$$

$$O \in \mathbb{R}^{T_q \times d_v}.$$

Use: Attention dimension check.

18.4 Causal Mask [Optional]

Masked scores

$$S_{\text{mask}} = S + M.$$

$$T_q = T_k = T, \quad M \in \mathbb{R}^{T \times T} \quad \text{for decoder self-attention.}$$

$$M_{ij} = \begin{cases} 0, & j \leq i, \\ -\infty, & j > i. \end{cases}$$

Masked attention weights

$$P = \text{softmax}(S_{\text{mask}}).$$

Use: Prevent attention to future tokens.

Notes: Used in autoregressive decoder self-attention.

18.5 Multi-Head Attention [Core]

Head projections

$$Q_r = XW_Q^{(r)}, \quad K_r = XW_K^{(r)}, \quad V_r = XW_V^{(r)}.$$

Use: Self-attention head projections.

$$W_Q^{(r)}, W_K^{(r)} \in \mathbb{R}^{d_{\text{model}} \times d_k}, \quad W_V^{(r)} \in \mathbb{R}^{d_{\text{model}} \times d_v}.$$

Attention head

$$\text{head}_r = \text{Attention}(Q_r, K_r, V_r), \quad r = 1, \dots, h.$$

$$\text{head}_r \in \mathbb{R}^{T \times d_v}.$$

Head concatenation

$$H = \text{Concat}(\text{head}_1, \dots, \text{head}_h).$$

$$H \in \mathbb{R}^{T \times (hd_v)}.$$

Use: Parallel attention heads.

Notes: Common setting: $d_k = d_v = d_{\text{model}}/h$.

18.6 Output Projection [Derived]

Multi-head output projection

$$\begin{aligned} \text{MHA}(X) &= HW_O. \\ W_O &\in \mathbb{R}^{hd_v \times d_{\text{model}}}, \quad \text{MHA}(X) \in \mathbb{R}^{T \times d_{\text{model}}}. \end{aligned}$$

Combined multi-head formula

$$\text{MultiHead}(Q, K, V) = \text{Concat}(\text{head}_1, \dots, \text{head}_h)W_O.$$

Use: Project concatenated heads back to model width.

18.7 Position-wise Feed-Forward Network [Derived]

Token-wise FFN

$$\begin{aligned} x &\in \mathbb{R}^{1 \times d_{\text{model}}}. \\ \text{FFN}(x) &= \phi(xW_1 + b_1^T)W_2 + b_2^T. \\ b_1^T &\in \mathbb{R}^{1 \times d_{\text{ff}}}, \quad b_2^T \in \mathbb{R}^{1 \times d_{\text{model}}}. \end{aligned}$$

Sequence FFN

$$\begin{aligned} \text{FFN}(X) &= \phi(XW_1 + \mathbf{1}b_1^T)W_2 + \mathbf{1}b_2^T. \\ \mathbf{1} &\in \mathbb{R}^{T \times 1}, \quad \mathbf{1}b_1^T \in \mathbb{R}^{T \times d_{\text{ff}}}, \quad \mathbf{1}b_2^T \in \mathbb{R}^{T \times d_{\text{model}}}. \\ W_1 &\in \mathbb{R}^{d_{\text{model}} \times d_{\text{ff}}}, \quad b_1 \in \mathbb{R}^{d_{\text{ff}} \times 1}. \\ W_2 &\in \mathbb{R}^{d_{\text{ff}} \times d_{\text{model}}}, \quad b_2 \in \mathbb{R}^{d_{\text{model}} \times 1}. \\ \text{FFN}(X) &\in \mathbb{R}^{T \times d_{\text{model}}}. \end{aligned}$$

Use: Token-wise feed-forward transformation.

Notes: Token-wise FFN uses row-vector convention; ϕ is often ReLU or GELU.

18.8 Positional Encoding [Derived]

Input with position information

$$\begin{aligned} X_{\text{pos}} &= X_{\text{emb}} + P_{\text{pos}}. \\ X_{\text{emb}}, P_{\text{pos}}, X_{\text{pos}} &\in \mathbb{R}^{T \times d_{\text{model}}}. \end{aligned}$$

Sinusoidal positional encoding

$$\begin{aligned} PE_{\text{pos}, 2i} &= \sin\left(\frac{\text{pos}}{10000^{2i/d_{\text{model}}}}\right). \\ PE_{\text{pos}, 2i+1} &= \cos\left(\frac{\text{pos}}{10000^{2i/d_{\text{model}}}}\right). \end{aligned}$$

Use: Inject token-position information.

18.9 Residual Connection and Layer Normalization [Optional]

Layer normalization

$$\begin{aligned}\text{LayerNorm}(x) &= \gamma \odot \frac{x - \mu}{\sqrt{\sigma^2 + \epsilon}} + \beta. \\ x &\in \mathbb{R}^{1 \times d_{\text{model}}}, \quad \gamma, \beta \in \mathbb{R}^{1 \times d_{\text{model}}}. \\ X &\in \mathbb{R}^{T \times d_{\text{model}}}, \quad (T \times d_{\text{model}}) \odot (1 \times d_{\text{model}}) \rightarrow T \times d_{\text{model}}.\end{aligned}$$

Post-norm residual

$$Y = \text{LayerNorm}(X + \text{Sublayer}(X)).$$

Pre-norm residual

$$Y = X + \text{Sublayer}(\text{LayerNorm}(X)).$$

Use: Residual connection with normalization.

Notes: Original Transformer commonly uses post-norm; many modern Transformers use pre-norm.

18.10 Transformer Block Pipeline [Core]

Encoder block, post-norm

$$\begin{aligned}H_1 &= \text{LayerNorm}(X + \text{MHA}(X)). \\ H_2 &= \text{LayerNorm}(H_1 + \text{FFN}(H_1)).\end{aligned}$$

Decoder block labels

$$X \rightarrow \text{MaskedMHA} \rightarrow \text{CrossAttention} \rightarrow \text{FFN}.$$

Cross-attention projections

$$\begin{aligned}Q &= X_{\text{dec}} W_Q, \quad K = X_{\text{enc}} W_K, \quad V = X_{\text{enc}} W_V. \\ X_{\text{dec}} &\in \mathbb{R}^{T_q \times d_{\text{model}}}, \quad X_{\text{enc}} \in \mathbb{R}^{T_k \times d_{\text{model}}}. \\ Q &\in \mathbb{R}^{T_q \times d_k}, \quad K \in \mathbb{R}^{T_k \times d_k}, \quad V \in \mathbb{R}^{T_k \times d_v}.\end{aligned}$$

Use: Transformer block computation pattern.

18.11 Transformer Symbol Table

Symbol	Meaning	Shape / Type
X	token representation matrix	$\mathbb{R}^{T \times d_{\text{model}}}$
X_{dec}	decoder token representations	$\mathbb{R}^{T_q \times d_{\text{model}}}$
X_{enc}	encoder token representations	$\mathbb{R}^{T_k \times d_{\text{model}}}$
X_{emb}	token embedding sequence	$\mathbb{R}^{T \times d_{\text{model}}}$
P_{pos}	positional encoding	$\mathbb{R}^{T \times d_{\text{model}}}$
X_{pos}	position-aware input	$\mathbb{R}^{T \times d_{\text{model}}}$
Q	query matrix	$\mathbb{R}^{T_q \times d_k}$
K	key matrix	$\mathbb{R}^{T_k \times d_k}$
V	value matrix	$\mathbb{R}^{T_k \times d_v}$
S	attention scores	$\mathbb{R}^{T_q \times T_k}$
P	attention weights	$\mathbb{R}^{T_q \times T_k}$
O	attention output	$\mathbb{R}^{T_q \times d_v}$
T	sequence length	$\mathbb{N}$
T_q	query length	$\mathbb{N}$
T_k	key/value length	$\mathbb{N}$
d_{model}	model width	$\mathbb{N}$
d_k	query/key width	$\mathbb{N}$
d_v	value width	$\mathbb{N}$
h	number of heads	$\mathbb{N}$
H	concatenated multi-head output	$\mathbb{R}^{T \times (hd_v)}$
W_Q, W_K, W_V	projection matrices	see shape table
W_O	output projection	$\mathbb{R}^{hd_v \times d_{\text{model}}}$
W_1, W_2	FFN weights	see shape table
b_1, b_2	FFN biases	see shape table
$\mathbf{1}$	all-ones column vector	$\mathbb{R}^{T \times 1}$
d_{ff}	FFN hidden width	$\mathbb{N}$
M	causal mask	$\mathbb{R}^{T \times T}$ for decoder self-attention
γ, β	LayerNorm scale / shift	feature-wise parameters
ϵ	denominator stabilizer	scalar
ϕ	FFN nonlinearity	function

Notes: P denotes attention weights; P_{pos} denotes positional encoding. LayerNorm β is not Adam β .

18.12 Shape Reference Table

Quantity	Shape
X	$T \times d_{\text{model}}$
X_{dec}	$T_q \times d_{\text{model}}$
X_{enc}	$T_k \times d_{\text{model}}$
Q, K	$T \times d_k$
V	$T \times d_v$
Q (cross-attention)	$T_q \times d_k$
K, V (cross-attention)	$T_k \times d_k, T_k \times d_v$
S, P	$T_q \times T_k$
O	$T_q \times d_v$
head_r	$T \times d_v$
H	$T \times (hd_v)$
W_O	$hd_v \times d_{\text{model}}$
$\text{MHA}(X)$	$T \times d_{\text{model}}$
Q (batch)	$m \times T_q \times d_k$
K (batch)	$m \times T_k \times d_k$
V (batch)	$m \times T_k \times d_v$
Q (multi-head batch)	$m \times h \times T_q \times d_k$
K (multi-head batch)	$m \times h \times T_k \times d_k$
V (multi-head batch)	$m \times h \times T_k \times d_v$
W_1	$d_{\text{model}} \times d_{\text{ff}}$
W_2	$d_{\text{ff}} \times d_{\text{model}}$
b_1	$d_{\text{ff}} \times 1$
b_2	$d_{\text{model}} \times 1$
$\mathbf{1}$	$T \times 1$
$\mathbf{1}b_1^\top$	$T \times d_{\text{ff}}$
$\mathbf{1}b_2^\top$	$T \times d_{\text{model}}$
M	$T \times T$ for causal self-attention
$X_{\text{emb}}, P_{\text{pos}}, X_{\text{pos}}$	$T \times d_{\text{model}}$

19 ML Strategy Formula Appendix

19.1 Confusion Matrix Quantities

Binary classification count labels

$$\begin{aligned}
 TP &= \text{true positives}, & FP &= \text{false positives.} \\
 TN &= \text{true negatives}, & FN &= \text{false negatives.}
 \end{aligned}$$

Use: Binary classification count labels.

19.2 Classification Metrics

Accuracy

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN}.$$

Precision

$$\text{Precision} = \frac{TP}{TP + FP}.$$

Recall

$$\text{Recall} = \frac{TP}{TP + FN}.$$

F1 score

$$F_1 = \frac{2 \cdot \text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}}.$$

Equivalent F1

$$F_1 = \frac{2TP}{2TP + FP + FN}.$$

Use: Classification evaluation metrics.

19.3 Error Rate

Error from accuracy

$$\text{Error} = 1 - \text{Accuracy}.$$

Binary classification error

$$\text{Error} = \frac{1}{m} \sum_{i=1}^m \mathbf{1} \left[\hat{y}^{(i)} \neq y^{(i)} \right].$$

Use: Misclassification rate.

19.4 Bias / Variance Gap Labels

Error labels

$$E_{\text{train}} = \text{training error}, \quad E_{\text{dev}} = \text{development-set error}.$$

$$E_{\text{human}} = \text{human-level error}.$$

Avoidable bias gap

$$\Delta_{\text{bias}} = E_{\text{train}} - E_{\text{human}}.$$

Variance gap

$$\Delta_{\text{variance}} = E_{\text{dev}} - E_{\text{train}}.$$

Use: Bias / variance diagnostic gaps.

Notes: Replace E_{human} with a baseline error when human-level error is not applicable.

Notes: These are signed diagnostic gaps; interpretation depends on the chosen baseline and split definitions.

19.5 Data Mismatch Gap

Train-dev and dev errors

$$E_{\text{train-dev}} = \text{train-dev error}, \quad E_{\text{dev}} = \text{development-set error}.$$

Data mismatch gap

$$\Delta_{\text{mismatch}} = E_{\text{dev}} - E_{\text{train-dev}}.$$

Use: Distribution mismatch diagnostic gap.

Notes: These are signed diagnostic gaps; interpretation depends on the chosen baseline and split definitions.

19.6 Error Analysis Fraction

Category error fraction

$$r_c = \frac{N_c}{N_{\text{error}}}.$$

N_c = number of errors in category c , N_{error} = total number of analyzed errors.

Use: Fraction of errors explained by category c .

19.7 Train / Dev / Test Split Notation

Dataset splits

$$\mathcal{D}_{\text{train}}, \quad \mathcal{D}_{\text{dev}}, \quad \mathcal{D}_{\text{test}}.$$

Split sizes

$$m_{\text{train}} = |\mathcal{D}_{\text{train}}|, \quad m_{\text{dev}} = |\mathcal{D}_{\text{dev}}|, \quad m_{\text{test}} = |\mathcal{D}_{\text{test}}|.$$

Empirical risk on split S

$$\hat{R}_S(f) = \frac{1}{m_S} \sum_{(x^{(i)}, y^{(i)}) \in \mathcal{D}_S} \mathcal{L}(f(x^{(i)}), y^{(i)}).$$

Use: Dataset split and empirical-risk notation.

19.8 ML Strategy Symbol Table

Symbol	Meaning	Type
TP	true positives	count
FP	false positives	count
TN	true negatives	count
FN	false negatives	count
Accuracy	correct classification fraction	scalar
Precision	positive prediction correctness	scalar
Recall	positive class coverage	scalar
F_1	harmonic mean of precision and recall	scalar
E_{train}	training error	scalar
E_{dev}	development-set error	scalar
E_{test}	test-set error	scalar
E_{human}	human-level error	scalar
$E_{\text{train-dev}}$	train-dev error	scalar
Δ_{bias}	avoidable bias gap	scalar
Δ_{variance}	variance gap	scalar
Δ_{mismatch}	data mismatch gap	scalar
N_c	errors in category c	count
N_{error}	analyzed errors	count
r_c	error-analysis fraction	scalar
$\mathcal{D}_{\text{train}}$	training split	dataset
$\mathcal{D}_{\text{dev}}$	development split	dataset
$\mathcal{D}_{\text{test}}$	test split	dataset
$\hat{R}_S(f)$	empirical risk on split S	scalar

20 Shape Reference Tables

20.1 Dense Layer Shapes

Quantity	Shape
$X = A^{[0]}$	$n_x \times m$
$A^{[l-1]}$	$n^{[l-1]} \times m$
$W^{[l]}$	$n^{[l]} \times n^{[l-1]}$
$b^{[l]}$	$n^{[l]} \times 1$
$Z^{[l]}, A^{[l]}$	$n^{[l]} \times m$
Y	$n_y \times m$

20.2 Logistic Regression Shapes

Quantity	Shape
$x^{(i)}$	$n_x \times 1$
X	$n_x \times m$
w	$n_x \times 1$
b	$\mathbb{R}$
Z, A, Y	$1 \times m$
dZ	$1 \times m$
dw	$n_x \times 1$
db	$\mathbb{R}$

20.3 Backpropagation Shapes

Quantity	Shape
$dA^{[l]}$	$n^{[l]} \times m$
$dZ^{[l]}$	$n^{[l]} \times m$
$dW^{[l]}$	$n^{[l]} \times n^{[l-1]}$
$db^{[l]}$	$n^{[l]} \times 1$
$dA^{[l-1]}$	$n^{[l-1]} \times m$

20.4 Optimizer Accumulator Shapes

Quantity	Shape
$u_{dW^{[l]},t}$	$n^{[l]} \times n^{[l-1]}$
$u_{db^{[l]},t}$	$n^{[l]} \times 1$
$s_{dW^{[l]},t}$	$n^{[l]} \times n^{[l-1]}$
$s_{db^{[l]},t}$	$n^{[l]} \times 1$
$m_{dW^{[l]},t}, v_{dW^{[l]},t}$	$n^{[l]} \times n^{[l-1]}$
$m_{db^{[l]},t}, v_{db^{[l]},t}$	$n^{[l]} \times 1$

Use: Momentum, RMSProp, and Adam accumulator shapes.

20.5 CNN Shapes

Quantity	Shape / Formula
$A^{[l-1]}$	$m \times n_H^{[l-1]} \times n_W^{[l-1]} \times n_C^{[l-1]}$
$W^{[l]}$	$f^{[l]} \times f^{[l]} \times n_C^{[l-1]} \times n_C^{[l]}$
$b^{[l]}$	$1 \times 1 \times 1 \times n_C^{[l]}$
$A^{[l]}$	$m \times n_H^{[l]} \times n_W^{[l]} \times n_C^{[l]}$
$n_H^{[l]}$	$\left\lfloor \frac{n_H^{[l-1]} + 2p^{[l]} - f^{[l]}}{s^{[l]}} \right\rfloor + 1$
$n_W^{[l]}$	$\left\lfloor \frac{n_W^{[l-1]} + 2p^{[l]} - f^{[l]}}{s^{[l]}} \right\rfloor + 1$
Pooling output	$m \times n_H^{[l]} \times n_W^{[l]} \times n_C^{[l-1]}$
Pooling channels	$n_C^{[l]} = n_C^{[l-1]}$

Use: NHWC CNN shape convention.

20.6 RNN / GRU / LSTM Shapes

Quantity	Shape
$x^{(t)}$	$n_x \times m$
$a^{(t)}$	$n_a \times m$
$c^{(t)}$	$n_a \times m$
$\hat{y}^{(t)}$	$n_y \times m$
W_a	$n_a \times (n_a + n_x)$
b_a	$n_a \times 1$
W_y	$n_y \times n_a$
b_y	$n_y \times 1$
$W_u, W_r, W_{\tilde{c}}$ (GRU)	$n_a \times (n_a + n_x)$
$b_u, b_r, b_{\tilde{c}}$ (GRU)	$n_a \times 1$
$W_f, W_i, W_{\tilde{c}}, W_o$ (LSTM)	$n_a \times (n_a + n_x)$
$b_f, b_i, b_{\tilde{c}}, b_o$ (LSTM)	$n_a \times 1$
$a_{\text{bi}}^{(t)}$	$2n_a \times m$

20.7 Embedding and Language Model Shapes

Quantity	Shape
o_w	$ V \times 1$
$O^{(t)}$	$ V \times m$
E	$d_e \times V $
e_w	$d_e \times 1$
$E^{(t)}$	$d_e \times m$
$a^{(t)}$	$n_a \times m$
W_o	$ V \times n_a$
b_o	$ V \times 1$
$z^{(t)}, \hat{y}^{(t)}$	$ V \times m$
E_{row}	$ V \times d_e$

20.8 Attention and Seq2Seq Shapes

Quantity	Shape
$a^{(t')}$	$n_a \times m$
$s^{(t)}$	$n_s \times m$
$e^{(t,t')}$	$1 \times m$
$\alpha^{(t,t')}$	$1 \times m$
$c^{(t)}$	$n_a \times m$
W_s	$n_e \times n_s$
W_a	$n_e \times n_a$
v_a	$n_e \times 1$
$\hat{y}^{(t)}$	$n_y \times m$
W_y	$n_y \times n_s$
b_y	$n_y \times 1$
W_o	$n_y \times (n_s + n_a)$
b_o	$n_y \times 1$
$\begin{bmatrix} s^{(t)} \\ c^{(t)} \end{bmatrix}$	$(n_s + n_a) \times m$

20.9 Transformer Shapes

Quantity	Shape
X	$T \times d_{\text{model}}$
Q, K	$T \times d_k$
V	$T \times d_v$
X_{dec}	$T_q \times d_{\text{model}}$
X_{enc}	$T_k \times d_{\text{model}}$
Q	$T_q \times d_k$
K	$T_k \times d_k$
V	$T_k \times d_v$
S, P	$T_q \times T_k$
O	$T_q \times d_v$
head_r	$T \times d_v$
H	$T \times (hd_v)$
W_O	$hd_v \times d_{\text{model}}$
$\text{MHA}(X)$	$T \times d_{\text{model}}$
W_1	$d_{\text{model}} \times d_{\text{ff}}$
W_2	$d_{\text{ff}} \times d_{\text{model}}$
$\mathbf{1}$	$T \times 1$
$\mathbf{1}b_1^\top$	$T \times d_{\text{ff}}$
$\mathbf{1}b_2^\top$	$T \times d_{\text{model}}$
Q (batch)	$m \times T_q \times d_k$
K (batch)	$m \times T_k \times d_k$
V (batch)	$m \times T_k \times d_v$
Q (multi-head batch)	$m \times h \times T_q \times d_k$
K (multi-head batch)	$m \times h \times T_k \times d_k$
V (multi-head batch)	$m \times h \times T_k \times d_v$

20.10 Framework Layout Mapping

Convention	Shape
Course dense formula	$X_{\text{dense}} \in \mathbb{R}^{n_x \times m}$
CNN NHWC	$X_{\text{CNN}} \in \mathbb{R}^{m \times H \times W \times C}$
PyTorch CNN NCHW	$X_{\text{CNN}} \in \mathbb{R}^{m \times C \times H \times W}$
Transformer batch-first	$X_{\text{Transformer}} \in \mathbb{R}^{m \times T \times d_{\text{model}}}$
Transformer formula-first	$X_{\text{Transformer}} \in \mathbb{R}^{T \times d_{\text{model}}}$

Use: Framework layout mapping.